

Unity를 이용한 2D 게임프로그래밍

동명대학교 게임학부

Lecture 9--10

Unity C# 프로그래밍

스크립트 기초에서 핵심 기능 체험까지

강영민

동명대학교 게임학부

대상 독자

이미 일반 C# 프로그래밍의 기초(클래스, 상속, 메서드, 제어문, 컬렉션 등)는 알고 있는 학생. Part I에서는 Unity 스크립트의 구조와 일반 C#과의 차이를 이해하고, Part II에서는 C#의 핵심 기능 6가지를 Unity라는 무대 위에서 직접 체험한다.

실습 환경: Unity Editor + Visual Studio Code (또는 Visual Studio)

Contents

I	Unity 스크립트로서의 C#	4
1	강의 개요	5
1.1	학습 목표	5
1.2	출발점: 일반 C#으로 게임을 만들 때 우리가 직접 한 일들	5
1.3	프레임워크와 라이브러리의 차이	6
2	Unity 스크립트의 기본 구조	6
2.1	가장 기본적인 Unity 스크립트	6
2.2	MonoBehaviour란 정확히 무엇인가?	7
2.3	일반 C# 프로그램과의 차이점	7
2.4	Scene(씬)이란?	8
3	MonoBehaviour의 생명 주기	8
3.1	핵심 생명 주기 메서드	8
3.2	Awake와 Start의 차이	8
3.3	Update와 FixedUpdate의 차이	9
3.4	Rigidbody2D와 물리 엔진	9
3.5	Time.deltaTime의 의미	10
4	컴포넌트 기반 설계	10
4.1	GameObject와 Component	10
4.2	상속(Inheritance) vs 조합(Composition)	11
4.3	GetComponent로 컴포넌트 접근	12
4.4	Transform: 모든 GameObject의 필수 컴포넌트	12
5	Inspector와 직렬화	13
5.1	public 필드는 Inspector에 노출된다	13
5.2	권장 패턴: [SerializeField] private	14
5.3	Inspector가 있다는 것의 의미	14
6	Unity 스크립트의 주요 고려 사항	15
6.1	(1) MonoBehaviour는 new로 만들지 마라	15
6.2	(2) Update에서 무거운 작업을 하지 마라	16
6.3	(3) Unity의 null은 일반 C#의 null과 다르다	17
6.4	(4) Coroutine: 시간에 따라 펼쳐지는 작업	17
6.5	(5) 직접 그리지 않는다	18
7	입력과 충돌 처리	19
7.1	한눈에 보는 비교: 일반 C#/GUI vs Unity	19
7.2	Input.GetKey 세 가지 변형	20
7.3	Collider와 충돌 콜백	20
7.4	Unity가 대신 해 주는 것들	21
8	실전 예제: 충돌하면 합쳐지는 공들	21
8.1	게임 컨셉과 물리 규칙	22
8.2	Hierarchy 설계	22
8.3	Prefab 설계: BallPrefab	23
8.4	스크립트 설계: 역할 분리	23

8.5	Ball.cs 구현	24
8.6	BallSpawner.cs 구현	25
8.7	Unity Editor에서 적용하기 (단계별)	26
8.8	동작 분석과 확장	27
9	단계별 도전 과제	27
9.1	Phase 1 --- 예제 변형 (기초)	28
9.2	Phase 2 --- 새 메커니즘 추가 (응용)	29
9.3	Phase 3 --- 새 게임 만들기 (종합 응용)	31
9.4	Phase 4 --- 자유 창작	32
10	이번 강의에서 배운 핵심 개념	33
II	Unity로 체험하는 C#의 핵심 기능들	34
11	강의 개요	35
11.1	이 강의의 주인공은 “C#”이다	35
11.2	학습 목표	35
11.3	왜 “Lab(실험실)” 형식인가?	35
11.4	이전 강의들과의 연결	35
12	프로젝트 준비	36
12.1	Unity 프로젝트 만들기	36
12.2	공통 도우미 스크립트: LogPanel.cs (선택)	36
13	Lab 1 --- enum과 switch 표현식	36
13.1	문제	37
13.2	콘솔에서라면	37
13.3	Unity 데모	37
13.4	Unity 구현 가이드	38
13.5	여기서 배우는 C# 개념	39
14	Lab 2 --- 프로퍼티와 캡슐화	40
14.1	문제	40
14.2	콘솔에서라면	40
14.3	Unity 데모	41
14.4	Unity 구현 가이드	41
14.5	여기서 배우는 C# 개념	42
15	Lab 3 --- 인터페이스 (interface)	43
15.1	문제	43
15.2	콘솔에서라면	43
15.3	Unity 데모	44
15.4	Unity 구현 가이드	45
15.5	여기서 배우는 C# 개념	46
16	Lab 4 --- delegate, Action, event	47
16.1	문제	47
16.2	콘솔에서라면	47
16.3	Unity 데모	48
16.4	Unity 구현 가이드	49
16.5	여기서 배우는 C# 개념	50

17 Lab 5 --- 제네릭 List<T>와 LINQ 맛보기	51
17.1 문제	51
17.2 콘솔에서라면	51
17.3 Unity 데모	52
17.4 Unity 구현 가이드	53
17.5 여기서 배우는 C# 개념	53
18 Lab 6 --- Coroutine과 람다	55
18.1 문제	55
18.2 콘솔이라면 비슷하게	55
18.3 Unity 데모	55
18.4 Unity 구현 가이드	56
18.5 여기서 배우는 C# 개념	57
19 이번 강의에서 배운 C# 개념 정리	58
19.1 한눈에 보기	58
19.2 Unity는 우리에게 무엇을 해 주었는가?	58
20 단계별 도전 과제	58
20.1 Phase 1 --- 같은 Lab에 기능 한 줄 더	59
20.2 Phase 2 --- 두 Lab을 합치기	59
20.3 Phase 3 --- 자유 실험	59
21 이번 강의에서 배운 핵심 개념	60

Part I

Unity 스크립트로서의 C#

1. 강의 개요

1.1. 학습 목표

같은 C# 언어를 사용하지만, Unity 안에서 동작하는 스크립트는 일반 C# 프로그램과 작성 방식이 매우 다르다. 이번 강의에서는 그 차이점과 주의해야 할 점을 정리한다.

- 일반 C# 프로그램과 Unity 스크립트의 진입 방식 차이를 이해한다.
- MonoBehaviour와 그 생명 주기 메서드(Awake, Start, Update 등)의 호출 시점을 파악한다.
- Unity의 컴포넌트 기반 설계와 GameObject/Component 모델을 이해한다.
- Unity의 직렬화(Serialization)와 Inspector 노출 규칙을 안다.
- Unity 스크립트 작성 시 흔한 실수와 성능 함정을 피할 수 있다.

1.2. 출발점: 일반 C#으로 게임을 만들 때 우리가 직접 한 일들

콘솔 또는 GUI 응용 프로그램(예: Windows Forms, WPF)으로 간단한 2D 게임을 직접 만든다고 상상해 보자. 다음을 모두 우리가 직접 작성해야 한다.

⚙️ 일반 C# 게임의 일반적 골격

```
Main()                                // 진입점
|
v
GameWindow / GameForm
|--- Timer (예: 16ms 마다 OnUpdate 호출)
|--- OnUpdate() : 위치 갱신, 충돌 검사
|--- OnDraw()   : Graphics 객체로 직접 그리기
|--- OnKeyDown() : 입력 추적
```

이 골격에는 다음 코드가 모두 “우리 손으로” 들어간다.

- 진입점 Main 작성 --- 어디서 시작하고, 무엇을 띄울 것인가.
- 타이머/루프 직접 설치 --- 16ms 간격으로 무엇을 호출할 것인가.
- 화면 갱신 요청 --- “다시 그려라”라고 명시적으로 호출.
- 직접 그리기 --- Graphics.FillRectangle, DrawImage 등을 매 프레임.
- 입력 추적 --- 현재 어떤 키가 눌러 있는지 직접 관리.
- 수동 충돌 판정 --- Math.Abs, 좌표 비교.

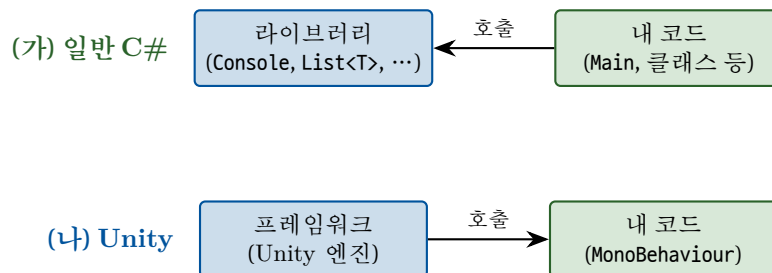
🔧 Unity의 출발점

Unity에서는 위 골격이 이미 만들어져 있다. Main을 작성할 필요도, 타이머를 만들 필요도, 화면 갱신을 호출할 필요도 없다. 대신 우리는 MonoBehaviour를 상속한 클래스를 작성하고 Update() 메서드만 채워 넣으면 된다. 나머지는 Unity 엔진이 알아서 호출해 준다.

1.3. 프레임워크와 라이브러리의 차이

🎬 할리우드 원칙

일반 C# 프로그램은 우리가 라이브러리를 호출한다 (→ `Console.WriteLine`).
 Unity 스크립트는 Unity 프레임워크가 우리를 호출한다 (→ `Update()`).
 이 “역전된 호출 관계”를 **IoC(Inversion of Control)** 또는 **할리우드 원칙** (“Don’t call us, we’ll call you”)이라고 부른다. 프레임워크가 일정한 약속(예: “`Update`라는 이름의 메서드는 매 프레임 호출함”)을 정해 두면, 우리는 그 약속에 맞춰 메서드를 작성하기만 하면 된다.



2. Unity 스크립트의 기본 구조

2.1. 가장 기본적인 Unity 스크립트

Unity Editor에서 “Create → MonoBehaviour Script”로 새 스크립트를 만들면 다음과 같은 코드가 자동으로 생성된다.

```

1 using UnityEngine;
2
3 public class HelloUnity : MonoBehaviour
4 {
5     void Start()
6     {
7         Debug.Log("Hello, Unity!");
8     }
9
10    void Update()
11    {
12        // Called every frame
13    }
14 }
  
```

이 짧은 코드 안에 **Unity 스크립트의 모든 핵심**이 들어 있다. 한 줄씩 살펴본다.

🔑 한 줄씩 분해

- `using UnityEngine;` --- Unity의 모든 기본 클래스를 담은 네임스페이스. `MonoBehaviour`, `GameObject`, `Transform`, `Debug` 등이 모두 여기에 있다.
- `public class HelloUnity` --- 클래스 이름은 파일 이름과 반드시 같아야 한다 (`HelloUnity.cs`). 다르면 Unity가 컴포넌트로 인식하지 못한다.
- `: MonoBehaviour` --- Unity가 인식하는 “스크립트 컴포넌트”가 되려면 반드시 이 클래스를 상속해야 한다.
- `void Start()` --- 게임 오브젝트가 활성화된 후 단 한 번 호출된다.
- `void Update()` --- 매 프레임마다 Unity가 자동으로 호출한다.

- `Debug.Log(...)` --- Unity의 Console 창에 메시지를 출력한다 (`Console.WriteLine`의 Unity 버전).

2.2. MonoBehaviour란 정확히 무엇인가?

`MonoBehaviour`는 Unity가 미리 만들어 둔 기반(base) 클래스이다. 이 클래스는 다음 두 가지를 가지고 있다:

1. 미리 정의된 “빈” 메서드들 --- `Awake`, `Start`, `Update`, `OnCollisionEnter` 등 약 30개. 기본 구현은 “아무 것도 안 함”이다.
2. 유용한 속성들 --- `transform`, `gameObject`, `enabled`, `name` 등.

우리가 `class HelloUnity : MonoBehaviour`라고 쓰면, 이 약속된 메서드 중 필요한 것만 골라서 재정의하는 셈이다.

⚠ 주의: override 키워드를 쓰지 않는다

일반 C#에서는 부모의 `virtual` 메서드를 자식에서 `override`로 표시해야 한다. Unity의 생명 주기 메서드는 `override`를 쓰지 않는다.

이는 Unity가 메서드 이름만 보고 리플렉션으로 찾아 호출하는 방식이기 때문이다. 메서드 이름의 철자가 틀리면 그냥 호출되지 않을 뿐 컴파일 에러는 발생하지 않으니 주의하라 (`Updaet`, `start`, `Awak` 같은 실수가 디버깅을 어렵게 만든다).

2.3. 일반 C# 프로그램과의 차이점

	일반 C# 프로그램	Unity 스크립트
진입점	<code>Main()</code> 메서드	없음 (Unity 엔진이 호출)
실행 단위	<code>.exe</code> 파일	<code>GameObject</code> 에 붙는 컴포넌트
객체 생성	<code>new MyClass()</code>	<code>AddComponent<MyClass>()</code>
필수 상속	없음	<code>MonoBehaviour</code>
파일 이름 규칙	자유	클래스 이름과 동일해야 함
프로그램 종료	<code>Main</code> 끝나면 종료	사용자가 정지 버튼 누를 때까지

i “Main이 없다”는 게 무슨 뜻인가?

일반 C# 프로그램에서 `Main`은 “프로그램의 시작점”이었다. Unity에서는 `Main`을 작성하지 않는다. Unity Editor에서 재생(Play) 버튼을 누르면 Unity 엔진이 다음 일을 자동으로 한다:

1. 현재 `Scene`에 있는 모든 `GameObject`를 찾는다.
2. 각 `GameObject`에 붙어 있는 `MonoBehaviour` 스크립트를 찾는다.
3. 각 스크립트의 `Awake` → `OnEnable` → `Start`를 차례로 호출.
4. 매 프레임마다 모든 활성 스크립트의 `Update`를 호출.

즉, Unity 엔진 자체가 거대한 `Main`이라고 생각하면 된다. 우리는 그 안에서 “특정 시점에 실행되어야 할 코드”만 약속된 이름의 메서드(`Start`, `Update` 등)에 작성한다.

2.4. Scene(씬)이란?

Scene은 “한 화면을 구성하는 GameObject들의 모음”이다. 하나의 게임은 보통 여러 개의 Scene으로 구성된다:

- TitleScene --- 타이틀 화면 (로고, 시작 버튼)
- StageScene --- 게임 플레이 화면 (플레이어, 적, UI)
- EndingScene --- 엔딩 화면

Scene은 파일로 저장된다 (.unity 확장자). 저장된 Scene을 SceneManager.LoadScene으로 불러오면 이전 Scene의 GameObject들이 사라지고 새 Scene의 GameObject들이 생성된다.

⇄ 일반 GUI 프로그램과의 비교

일반 GUI 프로그램에서 “타이틀 화면 → 게임 화면”을 구현하려면 같은 윈도우 안에서 화면을 “갈아 끼우는” 코드를 직접 짜야 한다. Unity는 **Scene** 단위로 코드를 분리할 수 있어 자연스럽게 화면별 책임이 나뉜다.

3. MonoBehaviour의 생명 주기

Unity 스크립트는 시간 순서대로 호출되는 여러 메서드를 가진다. 이를 생명 주기(Lifecycle) 또는 이벤트 함수(Event Functions)라고 한다.

3.1. 핵심 생명 주기 메서드

메서드

Awake()	오브젝트가 생성될 때 한 번 (비활성 상태)
OnEnable()	오브젝트가 활성화될 때
Start()	첫 Update 직전,
Update()	매 프레임 호출 (보통 초기화)
FixedUpdate()	고정 시간 간격(기본 0.02초)마다 호출
LateUpdate()	모든 Update가 끝난 후 호출 (카메라)
OnDisable()	오브젝트가 비활성화될 때
OnDestroy()	오브젝트가 삭제될 때

3.2. Awake와 Start의 차이

학생들이 가장 자주 헷갈리는 부분이다. 둘 다 “초기화”에 사용되지만 호출 시점이 다르다.

```

1 using UnityEngine;
2
3 public class LifecycleDemo : MonoBehaviour
4 {
5     void Awake()
6     {
7         // Called immediately when this object is created.
8         // Use for: initializing this object's own state.
9         Debug.Log("Awake");
10    }
11
12    void Start()
13    {

```

```

14     // Called just before the first Update.
15     // Use for: accessing OTHER objects (they all have Awake'd by now).
16     Debug.Log("Start");
17 }
18
19 void Update()
20 {
21     // Called every frame.
22 }
23 }

```

❓ 왜 두 개로 나뉘어 있는가?

Scene에 A, B, C 세 오브젝트가 있다고 하자. 호출 순서는 다음과 같다:

1. A.Awake() → B.Awake() → C.Awake() --- 이 시점에 모든 오브젝트의 “기본 준비”가 끝남.
2. A.Start() → B.Start() → C.Start() --- 이 시점에 다른 오브젝트가 이미 Awake되어 있음이 보장됨.
3. A.Update() → B.Update() → C.Update() → ...

규칙

- Awake: 자기 자신의 변수 초기화, 같은 GameObject의 컴포넌트 캐싱
- Start: 다른 오브젝트와 상호작용해야 하는 초기화

3.3. Update와 FixedUpdate의 차이

	Update	FixedUpdate
호출 간격	프레임마다 (불규칙)	고정 (기본 0.02초, 50회/초)
용도	입력, 일반 로직, 비물리 이동	Rigidbody 등 물리 연산
시간 변수	Time.deltaTime	Time.fixedDeltaTime

✓ 언제 어느 것을 써야 하나?

- 키보드 입력 검사 → Update
(입력은 프레임 단위로 받는 것이 자연스럽다)
- transform.position을 직접 바꾸는 이동 → Update
- Rigidbody.AddForce, Rigidbody.velocity 변경 → FixedUpdate
(물리 엔진이 고정 시간 간격으로 동작하기 때문)
- 카메라가 캐릭터를 따라가는 코드 → LateUpdate
(캐릭터의 Update가 모두 끝난 후 카메라가 따라가야 끊김 없음)

3.4. Rigidbody2D와 물리 엔진

Rigidbody(2D는 Rigidbody2D)는 “이 오브젝트는 물리 법칙의 영향을 받는다”라고 알리는 컴포넌트이다. 이 컴포넌트가 붙은 오브젝트에는 다음이 자동으로 적용된다:

- 중력 --- 아래로 자유 낙하

- 관성 --- 한번 움직이면 마찰이 없는 한 계속 움직임
- 충돌 반응 --- 다른 Collider에 부딪히면 튕겨 나감

⇄ 일반 C#과의 비교

일반 C#에서 “공이 벽에서 튀어 나오기”를 직접 코딩한다면

```
if (x < 0 || x > maxX) dx = -dx;
```

같은 검사를 우리가 매 프레임 짜야 한다.

Unity에서는 Rigidbody2D와 Collider2D, 그리고 “Bouncy” Physics Material 2D만 설정하면 공식이 필요 없이 알아서 튕긴다.

왜 FixedUpdate에서 다뤄야 하나? 물리 엔진은 정확한 시뮬레이션을 위해 고정된 시간 간격으로 계산을 해야 한다. 프레임 속도(가변)에 맞춰 물리를 계산하면, 빠른 컴퓨터에서는 공이 더 빨리 떨어지고 느린 컴퓨터에서는 천천히 떨어지는 이상한 현상이 생긴다. FixedUpdate는 항상 0.02초 간격으로 호출되므로 어떤 기기에서도 동일한 물리 결과가 나온다.

3.5. Time.deltaTime의 의미

Update는 프레임마다 호출되지만, 프레임 간격이 일정하지 않다. 어떤 프레임은 1/60초, 어떤 프레임은 1/30초가 걸릴 수 있다.

```
1 void Update()
2 {
3     // Wrong: speed depends on frame rate
4     transform.position += new Vector3(0.1f, 0, 0);
5
6     // Correct: speed independent of frame rate (5 units per second)
7     transform.position += new Vector3(5f * Time.deltaTime, 0, 0);
8 }
```

i Time.deltaTime이란?

Time.deltaTime은 “이전 프레임으로부터 지금까지 경과한 시간(초)”이다.

- 60fps → Time.deltaTime ≈ 0.0167초
- 30fps → Time.deltaTime ≈ 0.0333초

이동량 = 속도 × 시간 이므로, 속도 * Time.deltaTime을 더하면 프레임 속도와 무관하게 일정한 속도로 움직인다.

4. 컴포넌트 기반 설계

4.1. GameObject와 Component

Unity는 상속이 아니라 조합(Composition)으로 객체를 만든다.

⇄ 일반 C# vs Unity

일반 C# 방식 (상속/직접 작성):

```
class Player {
    public float X, Y;
    void Update(...) { ... }
```

```
void Draw(...) { ... }
}
```

Unity 방식 (컴포넌트 조합):

```
GameObject "Player"
  --- Transform      (position, rotation, scale)
  --- SpriteRenderer (image)
  --- Rigidbody2D    (physics)
  --- BoxCollider2D  (collision shape)
  --- PlayerController (our script)
```

GameObject 자체는 빈 컨테이너이다. “무엇을 할 수 있는가”는 거기에 붙어 있는 Component들이 결정한다. 이 사고방식은 처음에는 어색하지만, 익숙해지면 매우 강력하다.

4.2. 상속(Inheritance) vs 조합(Composition)

객체에 기능을 부여하는 두 가지 방식이 있다.

상속 방식 (전통 OOP) “Player는 Character이다”라는 is-a 관계.

```
class Character { ... }
class Movable : Character { ... }
class FlyingCharacter : Movable { ... }
class Player : FlyingCharacter { ... }
```

새 기능을 추가하려면 새 자식 클래스를 만들어야 한다. “걸을 수 있는 적”이 필요하다면 또 새 클래스, “날 수 있는 아이템”이 필요하다면 또 새 클래스 --- 계층이 점점 깊어지고 복잡해진다 (이른바 “상속 지옥”).

조합 방식 (Unity) “Player는 Mover를 가진다”라는 has-a 관계.

```
GameObject "Player"
  + Mover          // movement ability
  + Shooter        // shooting ability
  + HealthComponent // hp tracking
```

“걸을 수 있는 적”이 필요하다면 Enemy GameObject에 Mover만 붙이면 된다. 새 클래스가 필요 없다. 기능들이 레고 블록처럼 조립된다.

★ 한 줄 요약

상속이 “무엇이다(what it is)”라면, 조합은 “무엇을 가진다(what it has)”이다. Unity는 의도적으로 후자를 선택했다.

컴포넌트 모델의 장점

- 재사용성: Rigidbody2D는 플레이어든 적이든 누구든 붙일 수 있다.
- 유연성: 게임 도중에 컴포넌트를 추가/제거해서 동작을 바꿀 수 있다.
- 단순한 클래스: 각 컴포넌트는 하나의 일만 한다. “상속 지옥”(deep inheritance)이 발생하지 않는다.

4.3. GetComponent로 컴포넌트 접근

같은 GameObject에 붙은 다른 컴포넌트를 가져오려면 GetComponent<T>()를 사용한다.

```

1 using UnityEngine;
2
3 public class PlayerController : MonoBehaviour
4 {
5     Rigidbody2D rb;
6     SpriteRenderer sr;
7
8     void Awake()
9     {
10         // Cache references to other components on the same GameObject
11         rb = GetComponent<Rigidbody2D>();
12         sr = GetComponent<SpriteRenderer>();
13     }
14
15     void Update()
16     {
17         // Use the cached references
18         if (Input.GetKeyDown(KeyCode.Space))
19         {
20             rb.velocity = new Vector2(0, 5);
21             sr.color = Color.red;
22         }
23     }
24 }

```

⚠ 왜 Awake에서 캐싱하는가?

GetComponent<T>()는 매번 호출될 때마다 컴포넌트 목록을 검색한다. Update에서 매 프레임 호출하면 성능에 손해이다.

```

// BAD: searches every frame (60+ times per second)
void Update() {
    GetComponent<Rigidbody2D>().velocity = ...;
}

// GOOD: search once, reuse the reference
Rigidbody2D rb;
void Awake() { rb = GetComponent<Rigidbody2D>(); }
void Update() { rb.velocity = ...; }

```

이 패턴은 Unity 스크립트의 거의 모든 곳에서 사용된다. “Update에서 GetComponent를 호출하지 마라”는 가장 기본적인 성능 규칙이다.

4.4. Transform: 모든 GameObject의 필수 컴포넌트

모든 GameObject는 Transform 컴포넌트를 반드시 하나 가진다. 이는 위치(position), 회전(rotation), 크기(scale)를 담고 있다.

```

1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f;
4
5     void Update()
6     {

```

```

7      // Move 3 units per second to the right
8      transform.position += new Vector3(speed * Time.deltaTime, 0, 0);
9
10     // Rotate 90 degrees per second around the Z axis
11     transform.Rotate(0, 0, 90f * Time.deltaTime);
12 }
13 }

```

💡 transform은 어디서 왔나?

MonoBehaviour 는 내부적으로 transform 속성을 이미 가지고 있다. 별도로 GetComponent<Transform>() 을 호출할 필요 없이 그냥 transform이라고 쓰면 된다. 이는 너무 자주 쓰이는 컴포넌트이기 때문에 Unity가 미리 노출시켜 둔 것이다. 비슷하게 gameObject 속성도 “이 컴포넌트가 붙어 있는 GameObject”를 가리킨다.

📦 Vector3란 무엇인가?

Vector3는 “3차원 공간의 한 점 또는 방향”을 나타내는 구조체이다. 세 개의 float 값(x, y, z)으로 이루어져 있다.

```

Vector3 p = new Vector3(2f, 5f, 0f);
Debug.Log(p.x); // 2
Debug.Log(p.y); // 5

// Vector arithmetic
Vector3 a = new Vector3(1, 0, 0);
Vector3 b = new Vector3(0, 1, 0);
Vector3 c = a + b; // (1, 1, 0)
Vector3 d = a * 5f; // (5, 0, 0)

// Useful constants
Vector3.zero; // (0, 0, 0)
Vector3.up; // (0, 1, 0)
Vector3.right; // (1, 0, 0)

```

왜 x, y, z를 따로 두지 않고 Vector3로 묶나?

- 한 단위로 다루기 위해 --- transform.position = new Vector3(...) 한 줄로 위치를 통째로 설정.
- 벡터 연산 지원 --- +, -, * 같은 연산자가 의미 그대로 작동.

2D 게임에서는 z를 0으로 두거나 Vector2(x, y만 있음)를 쓴다.

5. Inspector와 직렬화

5.1. public 필드는 Inspector에 노출된다

Unity는 스크립트의 필드를 자동으로 Inspector에 표시한다. 이를 통해 코드를 수정하지 않고도 값을 바꿀 수 있다.

```

1 public class Mover : MonoBehaviour
2 {
3     public float speed = 3f; // Shown in Inspector
4     public Color color = Color.red; // Shown in Inspector

```

```

5 public GameObject target; // Shown in Inspector (drag-and-drop)
6
7 private int hiddenValue = 10; // NOT shown in Inspector
8 }

```

이는 일반 C# 프로그램에서는 보지 못한 새로운 개념이다. “디자이너는 코드를 모르지만 값은 조정할 수 있어야 한다”는 게임 개발 현장의 요구가 반영된 것이다.

Inspector 노출 규칙

- public 필드 → 자동 노출 (단, 직렬화 가능한 타입이어야 함)
- private 필드 → 노출되지 않음
- [SerializeField] private 필드 → 노출됨
- public + [HideInInspector] → 숨겨짐
- 프로퍼티(public int X { get; set; }) → 노출되지 않음

직렬화 가능한 타입: 기본형(int, float, ...), string, Vector2/3, Color, GameObject, Transform, MonoBehaviour를 상속한 클래스 등.

5.2. 권장 패턴: [SerializeField] private

public 필드는 Inspector에 노출되지만, 동시에 다른 스크립트에서도 자유롭게 접근 가능하다. 이는 캡슐화에 좋지 않다. Unity 커뮤니티에서는 다음 패턴을 권장한다:

```

1 public class Player : MonoBehaviour
2 {
3     // Editable in Inspector, but NOT accessible from other scripts
4     [SerializeField] private float speed = 3f;
5     [SerializeField] private int maxHp = 100;
6
7     // Read-only access from outside
8     public int CurrentHp { get; private set; }
9 }

```

💡 [SerializeField] 한 줄로 두 마리 토끼

- Inspector에서 조정 가능: 디자이너가 값을 바꿀 수 있다.
- 외부에서 접근 불가: private이므로 다른 클래스가 함부로 못 건드린다.

이는 OOP의 캡슐화 원칙을 Unity 환경에 맞춘 것이다. “필요한 것만 외부에 공개한다”는 OOP 원칙은 Unity에서도 그대로 유효하다.

5.3. Inspector가 있다는 것의 의미

✏️ 일반 C#과의 결정적 차이

일반 C# 프로그램에서 “플레이어 속도”를 0.5만큼 조정하려면 다음 사이클을 거쳐야 한다.
→ 코드 수정 → 재컴파일 → 실행 → 확인 → 다시 수정 ...

한 번 사이클에 수십 초가 걸린다.

Unity에서는 [SerializeField] float speed만 선언해 두면

→ Inspector에서 슬라이더 드래그 → 즉시 반영

심지어 플레이 중에도 값을 바꿔 가며 “느낌”을 잡을 수 있다. 이것이 게임 개발 생산성의 핵심 차이를 만든다.

6. Unity 스크립트의 주요 고려 사항

6.1. (1) MonoBehaviour는 new로 만들지 마라

일반 C# 클래스는 new MyClass()로 인스턴스를 만든다. MonoBehaviour는 다르다.

```
1 // WRONG: This compiles but Unity will not call Awake/Start/Update on it
2 PlayerController p = new PlayerController();
3
4 // CORRECT: attach to a GameObject
5 GameObject obj = new GameObject("Player");
6 PlayerController p = obj.AddComponent<PlayerController>();
7
8 // ALSO CORRECT: instantiate from a Prefab
9 GameObject p2 = Instantiate(playerPrefab);
```

⚠ 왜 new는 안 되는가?

MonoBehaviour는 “GameObject에 붙어서 동작하는 컴포넌트”이다. GameObject 없이 단독으로 존재하는 것은 의미가 없다.

new로 만들면 객체 자체는 생성되지만:

- Unity 엔진이 그 존재를 모른다 → Update가 호출되지 않는다.
- transform, gameObject 속성이 null이다.
- Destroy(this)를 호출해도 동작하지 않는다.

반드시 AddComponent<T>() 또는 Instantiate()를 사용해야 한다.

✂ AddComponent vs Instantiate vs Prefab

세 가지가 자주 같이 등장하지만 역할이 다르다.

AddComponent<T>() 이미 존재하는 GameObject에 컴포넌트 하나를 추가한다.

```
GameObject obj = new GameObject("Player");
obj.AddComponent<Rigidbody2D>(); // physics ability
obj.AddComponent<PlayerController>(); // our script
```

Instantiate(prefab) Prefab을 복제하여 Scene에 새 GameObject를 만든다. 이미 만들어진 “총알 프리팹”을 매번 복제해서 화면에 발사하는 식.

```
public GameObject bulletPrefab; // assigned in Inspector
GameObject b = Instantiate(bulletPrefab,
    transform.position,
    Quaternion.identity);
```

Prefab(프리팹)이란? “재사용 가능한 GameObject 템플릿”이다. 복잡한 GameObject(예: 모델 + 애니메이션 + 충돌체 + 스크립트가 모두 붙은 적 캐릭터)를 한 번 만들어 두고 .prefab

파일로 저장하면, 게임 도중 Instantiate로 무한히 복제 가능.

언제 어느 것을 쓰나?

- 단순한 일회성 객체 → new GameObject + AddComponent
- 자주 만들어지는 복잡한 객체(총알, 적, 아이템) → Prefab + Instantiate

6.2. (2) Update에서 무거운 작업을 하지 마라

Update는 초당 수십 수백 번 호출된다. 여기에 무거운 코드가 들어가면 게임 전체가 느려진다.

```

1 // BAD: searches the entire scene every frame
2 void Update()
3 {
4     GameObject enemy = GameObject.Find("Enemy");
5     enemy.transform.Translate(Vector3.left * Time.deltaTime);
6 }
7
8 // GOOD: find once, reuse
9 GameObject enemy;
10 void Start()
11 {
12     enemy = GameObject.Find("Enemy");
13 }
14 void Update()
15 {
16     enemy.transform.Translate(Vector3.left * Time.deltaTime);
17 }
```

⚠ Update에서 피해야 할 것들

- GameObject.Find(...) --- Scene 전체 검색
- FindObjectOfType<T>() --- 모든 오브젝트 순회
- GetComponent<T>() --- 컴포넌트 목록 검색 (캐싱하라)
- new SomeClass() 반복 --- 매 프레임 객체 생성 시 가비지 컬렉션 부담
- Debug.Log(...) 남발 --- 의외로 비용이 크다. 디버깅 후 지워야 한다.

“찾는 것은 한 번, 사용하는 것은 매번”이 기본 원칙이다.

🗑 가비지 컬렉션(GC)이 게임에서 왜 문제인가?

C#은 사용하지 않는 객체의 메모리를 자동으로 정리해 준다. 이를 가비지 컬렉션(GC)이라고 한다. 편리한 기능이지만 게임에서는 골칫거리이다.

문제: 프레임 끊김 GC는 “메모리 청소”를 위해 프로그램을 잠시 멈춘다. 일반 프로그램에서는 무시할 수준이지만, 60fps 게임은 한 프레임에 16ms뿐이므로 GC가 5ms만 발생해도 “끊김(stutter)”이 눈에 띈다.

```
void Update()
```

```
{
    // BAD: creates new object every frame -> GC pressure
    var dir = new System.Collections.Generic.List<int>();
    /* ... */
}
```

Vector3는 구조체라 GC와 무관하지만, new List<>(), new string() 같은 참조 타입을 매 프레임 만드는 것은 GC를 자주 깨우게 한다.

해결: 미리 만들고 재사용

- 필드로 미리 객체를 만들어 둔다 (Awake/Start에서)
- 문자열 합치기는 string + 대신 StringBuilder 사용
- 자주 생성/파괴되는 오브젝트는 **Object Pooling**(객체 풀) 패턴 사용

이는 일반 C#에서는 거의 신경 안 쓰는 부분이지만, Unity에서는 기본 상식이다.

6.3. (3) Unity의 null은 일반 C#의 null과 다르다

UnityEngine.Object(즉 GameObject, Transform 등)에 대한 == null 비교는 실제로는 Unity가 오버라이드한 비교 연산이 수행된다.

```
1 GameObject obj = ...;
2 Destroy(obj); // Object is marked for destruction
3
4 // One frame later:
5 if (obj == null)           // TRUE -- Unity reports it as null
6     Debug.Log("destroyed");
7
8 // But the C# reference is technically not null!
9 object asObj = obj;
10 if (asObj == null)        // FALSE -- the reference still exists
11     ...
```

☠ “가짜 null”(Fake Null) 현상

Destroy()로 삭제된 Unity 오브젝트는 C# 입장에서는 여전히 참조가 살아 있지만, Unity가 operator==를 오버로드해서 == null이면 true를 반환하게 만들었다.

실용적 규칙 Unity 오브젝트는 항상 == null로 비교하라. ?. 같은 null 조건 연산자나 ?? null 합치기 연산자는 이 “가짜 null”을 인식하지 못하므로 의도와 다르게 동작할 수 있다.

```
// Subtle bug
target?.SomeMethod(); // Calls method on a destroyed object!

// Safe
if (target != null) target.SomeMethod();
```

6.4. (4) Coroutine: 시간에 따라 펼쳐지는 작업

“2초 기다린 후 폭발”같은 시간 흐름이 있는 작업을 Update에서 처리하려면 매우 복잡해진다. Unity는 이를 위해 코루틴(Coroutine)을 제공한다.

```

1 using System.Collections;
2 using UnityEngine;
3
4 public class Bomb : MonoBehaviour
5 {
6     void Start()
7     {
8         StartCoroutine(ExplodeAfterDelay());
9     }
10
11     IEnumerator ExplodeAfterDelay()
12     {
13         Debug.Log("Bomb planted");
14         yield return new WaitForSeconds(2f);
15         Debug.Log("BOOM!");
16         Destroy(gameObject);
17     }
18 }

```

📌 코루틴 한 줄 요약

IEnumerator를 반환하는 메서드 안에서 yield return으로 “잠시 멈춰라”라고 표시할 수 있다.

- yield return null; --- 다음 프레임까지 대기
- yield return new WaitForSeconds(2f); --- 2초 대기
- yield return new WaitUntil(() => isReady); --- 조건이 참이 될 때까지 대기

이는 “비동기 코드를 마치 동기처럼 쓰게 해 주는” Unity의 기능이다. 일반 C#의 async/await와 비슷한 개념이지만, Unity의 게임 루프와 잘 어울리도록 설계되어 있다.

❓ 일반 C#이라면 어떻게?

“2초 후 폭발”을 일반 C#으로 짜려면

- (a) Thread.Sleep(2000) → 프로그램이 멈춰서 입력도 못 받음
- (b) Timer 콜백 → 콜백 지옥, 상태 관리 복잡
- (c) async/await → 가능하지만 게임 루프와 잘 안 맞음

코루틴은 (1) 메인 스레드에서 안전하게 실행되고, (2) 메서드 안의 지역 변수가 자동 보존되며, (3) 게임 루프와 자연스럽게 동기화된다.

6.5. (5) 직접 그리지 않는다

일반 GUI 프로그램에서는 “Paint 이벤트”에서 Graphics 객체로 직접 도형을 그렸다. Unity에서는 그렇지 않다.

- 이미지는 SpriteRenderer 컴포넌트가 그린다.
- 3D 모델은 MeshRenderer가 그린다.
- UI 텍스트는 TextMeshProUGUI가 그린다.

우리 스크립트는 무엇을 그릴지 결정만 한다(위치 변경, 색상 변경, sprite 교체 등). 실제 그리기는 Unity의 렌더링 시스템이 담당한다.

예: 색상 바꾸기

```
public class ColorChanger : MonoBehaviour
{
    SpriteRenderer sr;

    void Awake() { sr = GetComponent<SpriteRenderer>(); }

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.R)) sr.color = Color.red;
        if (Input.GetKeyDown(KeyCode.G)) sr.color = Color.green;
    }
}
```

우리는 `sr.color = ...`로 “빨간색으로 그려”라고 지시만 한다. 실제 픽셀을 칠하는 것은 Unity의 렌더러이다.

★ 렌더러(Renderer) 컴포넌트 종류

컴포넌트	용도
SpriteRenderer	2D 이미지(스프라이트) 표시
MeshRenderer	3D 메쉬(모델) 표시
TextMeshProUGUI	UI 텍스트 (점수, HP 등)
LineRenderer	선/궤적 (총알 자국, 레이저)
ParticleSystem	파티클 효과 (불꽃, 연기)

GameObject에 위 컴포넌트를 붙이고 sprite/메시 등을 지정해 두면 끝. 이후로는 어떤 그리기 코드도 작성하지 않아도 매 프레임 자동으로 그려진다. 스크립트는 `transform.position`이나 `sr.color`를 바꿔서 “어디에/어떻게” 그릴지만 결정한다.

7. 입력과 충돌 처리

7.1. 한눈에 보는 비교: 일반 C#/GUI vs Unity

田 같은 일을 어떻게 하나

	일반 C#/GUI 프로그램	Unity
진입점	Main / Application.Run	없음 (엔진이 자동)
게임 루프	Timer/while 직접 작성	Update 자동 호출
프레임 보정	직접 시간 측정	Time.deltaTime
화면 갱신	Invalidate + Paint	자동 렌더링
그리기	Graphics.FillRectangle 등	SpriteRenderer
객체 모델	클래스 직접 작성	GameObject + Component
입력 처리	KeyDown 이벤트 핸들러	Input.GetKey 폴링
충돌 판정	Math.Abs(...) < N 직접	Collider + 콜백
값 조정	코드 수정 후 재컴파일	Inspector에서 즉시

7.2. Input.GetKey 세 가지 변형

일반 GUI 프로그래밍에서 “현재 어떤 키가 눌러 있는지”를 추적하려면 KeyDown/KeyUp 이벤트를 HashSet 같은 곳에 모아 직접 관리해야 했다. Unity에서는 그 작업이 한 줄로 끝난다. 다만 세 가지 변형이 있고, 차이를 정확히 알아야 한다.

메서드

Input.GetKey(K)

Input.GetKeyDown(K)

Input.GetKeyUp(K)

키가 눌러 있는 동안 매 프레임

키를 누른 그 프레임에 한 번만 true

키를 떼는 그 프레임에 한 번만 true

```
void Update()
{
    // Continuous movement: while held
    if (Input.GetKey(KeyCode.LeftArrow))
        transform.Translate(-5 * Time.deltaTime, 0, 0);

    // One-shot action: only the moment of press
    if (Input.GetKeyDown(KeyCode.Space))
        Shoot();
}
```

⚠ 흔한 실수

점프에 GetKey를 쓰면, Space 키를 길게 누르고 있는 동안 매 프레임 점프 코드가 실행되어 캐릭터가 미친 듯이 튀어 오른다. “한 번 누름 = 한 번 동작”인 행동에는 반드시 GetKeyDown을 써야 한다.

7.3. Collider와 충돌 콜백

일반 C#에서는 충돌을 직접 계산했다 ($\text{Math.Abs}(b.X - X) < 20$). Unity에서는 GameObject에 Collider(2D는 Collider2D)를 붙이면 충돌 검사를 엔진이 대신 해 주고, 결과를 콜백 메서드로 알려 준다.

Collider 종류

- BoxCollider2D --- 사각형 (간단, 빠름)
- CircleCollider2D --- 원형
- PolygonCollider2D --- 임의 다각형 (복잡, 느림)

```
void OnCollisionEnter2D(Collision2D c) {
    // when a Rigidbody Collider hits another Collider
    Debug.Log("Hit: " + c.gameObject.name);
}

void OnTriggerEnter2D(Collider2D c) {
    // when entering a "trigger" zone (no physics push)
    Debug.Log("Entered: " + c.gameObject.name);
}
```

🔗 Collision vs Trigger의 차이

- Collision --- “실제로 부딪히는” 충돌. 물리적으로 튕긴다 (벽, 바닥).
- Trigger --- “통과는 되지만 알림은 받는” 충돌. 안 튕긴다 (아이템 획득 영역, 트리거 게이트).

Collider의 **Is Trigger** 체크박스를 켜면 Trigger 모드가 된다.

7.4. Unity가 대신 해 주는 것들

충돌이 일어나면 자동으로 호출되는 메서드

★ 요약

Unity에서 “코드가 짧다”고 느껴지는 이유는, 우리가 작성하지 않아도 되는 부분이 매우 많기 때문이다:

- 진입점(Main), 메시지 루프, 창 생성
- 프레임 타이머, 더블 버퍼링, 화면 갱신
- 키보드/마우스/터치 입력의 저수준 처리
- 물리 시뮬레이션 (중력, 충돌, 마찰)
- 텍스처/메시 로딩, 셰이더, 렌더링 파이프라인
- 오디오 재생, 네트워크, 파일 시스템 추상화

대신 우리는 “내 게임의 규칙”만 쓰면 된다. 이것이 게임 엔진의 가치이고, Unity 스크립트가 일반 C#과 다른 모양을 가지는 이유이다.

8. 실전 예제: 충돌하면 합쳐지는 공들

지금까지 배운 개념을 한 번에 적용하는 작은 게임을 만든다. 화면에 여러 개의 공이 떠다니다가 벽에 부딪히면 튕기고, 공끼리 부딪히면 하나로 합쳐지는 시뮬레이션이다.

이 예제 하나에 다음 개념이 모두 들어 있다:

- MonoBehaviour, Awake/Start 초기화
- Rigidbody2D + Collider2D로 자동 물리/충돌
- OnCollisionEnter2D 충돌 콜백
- Prefab + Instantiate로 다수 객체 생성
- [SerializeField]로 Inspector 노출
- Destroy로 오브젝트 제거

8.1. 게임 컨셉과 물리 규칙

⚙ 동작 명세

1. 시작 시 화면에 **N개의 공**이 무작위 위치에 생성된다.
2. 각 공은 무작위 방향으로 일정 속도로 움직인다.
3. 공이 **벽**에 부딪히면 튕긴다 (운동 에너지 보존).
4. 공 두 개가 서로 충돌하면 하나로 합쳐진다.
 - 합쳐진 공의 **면적**은 두 공의 면적의 합 ($r_{new} = \sqrt{r_1^2 + r_2^2}$)
 - 합쳐진 공의 **질량**은 두 공의 질량의 합
 - 합쳐진 공의 **속도**는 운동량 보존 ($\vec{v}_{new} = \frac{m_1\vec{v}_1 + m_2\vec{v}_2}{m_1 + m_2}$)
 - 합쳐진 공의 **위치**는 질량 중심 위치

💡 왜 면적을 보존하는가?

2D에서 “공의 면적”은 3D의 “부피”에 해당한다. 물리적으로 두 물체가 합쳐질 때 부피(면적)가 보존된다고 보면 자연스럽다.

원의 면적은 πr^2 이므로, $\pi r_1^2 + \pi r_2^2 = \pi r_{new}^2$ 에서 $r_{new} = \sqrt{r_1^2 + r_2^2}$ 이다.

만약 단순히 $r_{new} = r_1 + r_2$ 로 하면 “공이 너무 빨리 커지는” 비현실적인 결과가 나온다.

8.2. Hierarchy 설계

Unity Editor의 **Hierarchy** 창에 만들 GameObject 구조는 다음과 같다.

🏠 Scene Hierarchy

Main Camera	(Unity가 기본 제공)
GameManager	(빈 GameObject)
+-- BallSpawner.cs	(스크립트 컴포넌트)
Walls	(빈 GameObject -- 그룹 정리용)
+-- WallTop	(BoxCollider2D + SpriteRenderer)
+-- WallBottom	
+-- WallLeft	
+-- WallRight	

공(Ball)은 Hierarchy에 미리 두지 않고 **Prefab**으로만 만들어 둔 다음 BallSpawner가 게임 시작 시에 Instantiate로 생성한다.

📁 빈 GameObject로 그룹을 만드는 이유

Walls라는 빈 GameObject 아래에 4개의 벽을 두면:

- Hierarchy가 시각적으로 정리된다.
- Walls.SetActive(false)로 4개 벽을 한 번에 비활성화할 수 있다.
- Walls.transform.position을 옮기면 4개 벽이 함께 이동한다.

이는 “관련 있는 것끼리 묶어 둔다”는 OOP 원칙을 Hierarchy에 적용한 것이다.

8.3. Prefab 설계: BallPrefab

공 하나에 필요한 컴포넌트는 다음과 같다.

컴포넌트	역할
Transform	위치/크기 (필수, 자동 포함)
SpriteRenderer	원 모양 이미지 표시
CircleCollider2D	원형 충돌 영역
Rigidbody2D	물리 (관성, 충돌 응답)
Ball.cs	우리가 작성할 스크립트

Rigidbody2D는 다음과 같이 설정한다:

- Gravity Scale = 0 --- 중력 없음 (위에서 떨어지지 않게)
- Linear Drag = 0 --- 공기 저항 없음 (계속 움직이게)
- Collision Detection = Continuous --- 빠른 공 통과 방지

또한 **Physics Material 2D**를 만들어 CircleCollider2D에 할당한다:

- Friction = 0 --- 마찰 없음
- Bounciness = 1 --- 완전 탄성 충돌 (속도 보존)

✔ 우리가 작성하는 코드는 얼마나 적은가?

공이 “무작위로 움직이고, 벽에 튕기고, 다른 공과 부딪히는” 동작 중 “무작위 초기 속도 부여”와 “공-공 충돌 시 합치기”만 우리가 코딩한다. 나머지(움직임, 벽 충돌, 튕김 각도)는 **Rigidbody2D + Collider2D + Physics Material**이 자동으로 한다.

같은 게임을 일반 C#/GUI로 짤다면 dx, dy로 직접 속도를 관리하고 if ($x < 0$) $dx = -dx$;로 반사를 직접 코딩해야 한다.

8.4. 스크립트 설계: 역할 분리

스크립트는 두 개로 나눈다.

📁 두 개의 스크립트

- Ball.cs --- 개별 공의 동작
 - 시작 시 무작위 방향으로 발사
 - 다른 공과 충돌하면 합치기
- BallSpawner.cs --- 전체 게임 관리
 - 시작 시 N개의 공을 생성

이는 “한 클래스는 하나의 일” 원칙을 따른 것이다. BallSpawner는 “공을 만든다”는 일만, Ball은 “공이 어떻게 행동하는가”만 안다.

⚠ 충돌 시 “누가 합치는가” 문제

공 A와 공 B가 충돌하면 OnCollisionEnter2D가 두 번 호출된다 (A에서 B를 만남, B에서 A를 만남). 양쪽이 모두 합치기 코드를 실행하면 Destroy가 두 번 일어나거나 합치기가 두 번

실행되는 버그가 생긴다.

해결: ID 비교로 “더 작은 쪽이 처리” `GameObject.GetInstanceID()`는 모든 오브젝트마다 고유한 정수이다. “ID가 더 작은 쪽”만 합치기 코드를 실행하면 정확히 한 번만 처리된다.

8.5. Ball.cs 구현

```

1 using UnityEngine;
2
3 public class Ball : MonoBehaviour
4 {
5     [SerializeField] private float initialSpeed = 3f;
6
7     private Rigidbody2D rb;
8
9     void Awake()
10    {
11        rb = GetComponent<Rigidbody2D>();
12    }
13
14    void Start()
15    {
16        // Random initial direction
17        float angle = Random.Range(0f, 360f) * Mathf.Deg2Rad;
18        Vector2 dir = new Vector2(Mathf.Cos(angle), Mathf.Sin(angle));
19        rb.velocity = dir * initialSpeed;
20    }
21
22    void OnCollisionEnter2D(Collision2D c)
23    {
24        // Try to find a Ball component on the other object
25        Ball other = c.gameObject.GetComponent<Ball>();
26
27        // If null, it was a wall - let physics handle it (bounce)
28        if (other == null) return;
29
30        // Process merge only once: the ball with smaller ID handles it
31        if (GetInstanceID() > other.GetInstanceID()) return;
32
33        Merge(other);
34    }
35
36    void Merge(Ball other)
37    {
38        // Current radii (sprite is 1x1 unit by default)
39        float r1 = transform.localScale.x * 0.5f;
40        float r2 = other.transform.localScale.x * 0.5f;
41
42        // Conservation of area: r_new^2 = r1^2 + r2^2
43        float rNew = Mathf.Sqrt(r1 * r1 + r2 * r2);
44
45        // Conservation of mass
46        float m1 = rb.mass;
47        float m2 = other.rb.mass;
48        float mNew = m1 + m2;
49    }

```

```

50     // Conservation of momentum
51     Vector2 vNew = (m1 * rb.velocity + m2 * other.rb.velocity) / mNew;
52
53     // Center of mass position
54     Vector2 p1 = transform.position;
55     Vector2 p2 = other.transform.position;
56     Vector2 pNew = (m1 * p1 + m2 * p2) / mNew;
57
58     // Apply to this ball
59     transform.position = pNew;
60     transform.localScale = Vector3.one * (rNew * 2f);
61     rb.mass = mNew;
62     rb.velocity = vNew;
63
64     // Remove the other ball
65     Destroy(other.gameObject);
66 }
67 }

```

🔍 코드 한 부분씩 살펴보기

- [SerializeField] private float initialSpeed --- Inspector에서 초기 속도를 조정할 수 있게 노출.
- rb = GetComponent<Rigidbody2D>() 를 Awake에서 --- 한 번만 검색해서 캐싱 (성능 규칙).
- Start에서 무작위 방향 --- 모든 공의 Awake가 끝난 후 호출되므로 모든 공이 동시에 동작 시작.
- c.gameObject.GetComponent<Ball>() --- 부딪힌 상대가 공인지 벽인지 판별. 벽에는 Ball.cs 가 없으므로 null이 반환되어 “벽 처리”로 분기.
- GetInstanceID() > other.GetInstanceID()로 한 번만 처리 --- “양쪽이 모두 합치는 버그” 방지.
- Destroy(other.gameObject) --- 다른 공을 컴포넌트가 아니라 **GameObject** 단위로 제거. Destroy(other)만 하면 컴포넌트만 사라지고 GameObject는 남는다.

8.6. BallSpawner.cs 구현

```

1 using UnityEngine;
2
3 public class BallSpawner : MonoBehaviour
4 {
5     [SerializeField] private GameObject ballPrefab;
6     [SerializeField] private int count = 10;
7     [SerializeField] private Vector2 areaMin = new Vector2(-7, -4);
8     [SerializeField] private Vector2 areaMax = new Vector2(7, 4);
9
10    void Start()
11    {
12        for (int i = 0; i < count; i++)
13        {
14            float x = Random.Range(areaMin.x, areaMax.x);
15            float y = Random.Range(areaMin.y, areaMax.y);
16            Vector3 pos = new Vector3(x, y, 0);

```

```

17
18         Instantiate(ballPrefab, pos, Quaternion.identity);
19     }
20 }
21 }

```

ballPrefab은 어떻게 채워지는가?

[SerializeField] private GameObject ballPrefab; 한 줄을 쓰면 Inspector에 Ball Prefab이라는 빈 슬롯이 생긴다. Project 창에서 만든 BallPrefab을 그 슬롯으로 드래그 앤 드롭하면 연결된다.

왜 이렇게 하나? 대안은 코드에서 Resources.Load("BallPrefab")로 직접 찾는 방법이지만:

- 문자열 오타에 취약 ("Ballprefab" 같은 실수)
- 프리팹 이름이 바뀌면 모든 코드를 고쳐야 함

Inspector 슬롯에 드래그하면 Unity가 참조 자체를 저장하므로 이름이 바뀌어도 자동으로 따라간다.

8.7. Unity Editor에서 적용하기 (단계별)

Step-by-Step 셋업

Step 1: 새 2D 프로젝트 생성

Unity Hub에서 "New Project" → "2D Core" 선택.

Step 2: 벽 만들기

- Hierarchy에서 "Create Empty"로 Walls 생성.
- Walls 아래에 "2D Object > Sprites > Square" 4개 생성.
- 각각 WallTop, WallBottom, WallLeft, WallRight로 이름 변경.
- Scale과 Position을 조정해 화면 가장자리에 배치.
- 각 벽에 Add Component → BoxCollider2D 추가.

Step 3: BallPrefab 만들기

- Hierarchy에서 "2D Object > Sprites > Circle" 생성, 이름 Ball.
- Add Component로 Rigidbody2D 추가 (→ Gravity Scale = 0으로 변경).
- Add Component로 CircleCollider2D 추가.
- Project 창에서 우클릭 → Create > 2D > Physics Material 2D, 이름 Bouncy, Friction = 0, Bounciness = 1.
- CircleCollider2D의 Material 슬롯에 Bouncy 드래그.
- Ball.cs 스크립트 작성 후 Add Component로 부착.
- Hierarchy의 Ball을 Project 창으로 드래그 → BallPrefab.prefab 생성.
- Hierarchy에 남은 Ball은 삭제 (Prefab만 남기면 됨).

Step 4: GameManager + Spawner

- Hierarchy에 빈 GameObject **GameManager** 생성.
- **BallSpawner.cs** 스크립트를 부착.
- Inspector의 **Ball Prefab** 슬롯에 **BallPrefab**을 드래그.

Step 5: 재생

- Editor 상단의 **Play** 버튼 클릭.
- 10개의 공이 무작위로 생성되어 튕기고, 부딪히면 큰 공 하나로 합쳐지는 것을 확인.

8.8. 동작 분석과 확장**田 무엇이 어디서 일어나는가****이벤트****누가 처리하나**

공 생성	BallSpawner.Start
초기 속도 설정	Ball.Start
매 프레임 위치 갱신	Rigidbody2D (자동)
벽 충돌 반사	Collider2D + Physics Material 2D (자동)
공-공 충돌 감지	OnCollisionEnter2D 콜백
합치기 계산 및 적용	Ball.Merge
사라진 공 제거	Destroy(other.gameObject)

코드는 약 60줄뿐이지만, 자동 처리되는 부분(렌더링, 물리, 충돌 검출)을 우리가 직접 짤다면 일반 C#/GUI 방식으로 수백 줄이 필요하다.

앞 절의 표가 보여주듯, 우리가 작성한 부분은 “무작위 초기 속도”와 “합치기 로직”뿐이다. 이제 이 코드 골격을 발판으로 학생들이 직접 만들어 볼 도전 과제를 다음 절에서 단계별로 제시한다.

9. 단계별 도전 과제

앞에서 만든 “공 합치기” 예제를 출발점으로 삼아, 학생들이 직접 만들어 볼 수 있는 도전 과제를 난이도 순서대로 제시한다. 모든 과제는 앞 예제의 **BallPrefab**과 두 스크립트를 그대로 재사용할 수 있게 설계되어 있다.

A 과제 진행 권장 순서

- **Phase 1**을 먼저 모두 끝낸다. 코드 수정량이 적고 즉각적인 시각/청각 피드백이 있어 Unity의 “Inspector 즉시 반영” 감각을 익히기 좋다.
- **Phase 2**는 새로운 컴포넌트(UI, AudioSource)나 코루틴이 들어간다. 본문에서 다룬 개념을 실제 코드에서 한 번씩 사용해 보는 단계.
- **Phase 3**는 “공 합치기”와는 다른 게임을 처음부터 만든다. 일반 C#과의 차이를 가장 강하게 체감하는 단계.
- **Phase 4**는 자유 창작. Phase 1~3의 어떤 요소든 조합해 본인만의 게임을 완성한다.

📁 제출 양식 (공통)

- 프로젝트 폴더 압축 --- Assets/, Packages/, ProjectSettings/만 포함. Library/, Temp/, Logs/, obj/는 반드시 제외 (용량 폭증).
- README.md --- 어떤 과제를 풀었는지, 사용한 키, 화면 캡처 1~2장.
- Demo.mp4 --- 30초 이내 동작 영상 (선택, 가능하면).
- Phase 별로 최소 1개 과제 이상 풀이를 권장한다.

9.1. Phase 1 --- 예제 변형 (기초)

기존 코드의 일부만 바꾸어 시각/청각 피드백을 추가하는 단계이다. “코드를 적게 고쳐도 결과가 크게 달라진다”는 Unity의 강점을 체감해 보자.

🎮 과제 1.1 --- 색상 평균으로 합치기

공이 합쳐질 때 새 공의 색이 두 공 색의 평균이 되도록 만들어라.
요구 사항

1. Ball.Start()에서 무작위 색으로 초기화한다.
2. Merge()에서 새 색을 두 공 색의 산술 평균으로 설정한다.

힌트

```
private SpriteRenderer sr;

void Awake() {
    rb = GetComponent<Rigidbody2D>();
    sr = GetComponent<SpriteRenderer>(); // cache
}

void Start() {
    sr.color = new Color(Random.value, Random.value, Random.value);
    /* ... existing velocity init ... */
}

// In Merge():
sr.color = (sr.color + other.sr.color) * 0.5f;
```

생각해 볼 점

Color끼리의 덧셈은 채널별 덧셈이다. 채널 값이 1.0을 넘으면 어떻게 보이는가? “곱셈”은 어떤 의미일까?

🎮 과제 1.2 --- 속도에 따라 색이 변하는 공

공의 현재 속력이 클수록 빨강게, 작을수록 파랑게 보이도록 하라.
요구 사항

- Update에서 rb.velocity.magnitude를 읽어 색을 갱신.
- 속도 0~maxSpeed를 0~1로 정규화하여 Color.Lerp 적용.

힌트

```
[SerializeField] private float maxSpeed = 5f;
```

```
void Update() {
    float t = Mathf.Clamp01(rb.velocity.magnitude / maxSpeed);
    sr.color = Color.Lerp(Color.blue, Color.red, t);
}
```

확장 합치기 직후 속도가 느려지는 경향(질량 평균)을 색으로 시각화하면 운동량 보존을 눈으로 확인할 수 있다.

🔊 과제 1.3 — 충돌 사운드

벽 충돌과 공-공 합치기에 서로 다른 효과음을 재생하라.
요구 사항

- BallPrefab에 AudioSource 컴포넌트 추가.
- [SerializeField] AudioClip wallHit, mergeHit;로 두 사운드 노출.
- OnCollisionEnter2D에서 상대가 Ball인지 벽인지에 따라 PlayOneShot 분기.

힌트

- 효과음은 .wav 또는 .mp3를 Assets/Audio/ 폴더에 넣고 Inspector 슬롯에 드래그.
- audioSource.PlayOneShot(clip); --- 한 번만 재생, 여러 번 겹쳐 재생 가능.

9.2. Phase 2 --- 새 메커니즘 추가 (응용)

새로운 컴포넌트를 도입하거나 게임 규칙 자체를 바꾸는 단계이다. Coroutine, UI 컴포넌트, 입력 처리를 직접 사용해 본다.

🖱️ 과제 2.1 — 마우스 클릭으로 공 추가

화면을 좌클릭하면 그 위치에 새 공이 한 개 생성되도록 하라.
요구 사항

- BallSpawner.Update()에 추가 로직 작성.
- 마우스의 화면 좌표(픽셀)를 월드 좌표로 변환해야 함.

힌트

```
void Update() {
    if (Input.GetMouseButtonDown(0)) {
        Vector3 mp = Input.mousePosition;
        // distance from camera to z=0 plane
        mp.z = -Camera.main.transform.position.z;
        Vector3 world = Camera.main.ScreenToWorldPoint(mp);
        world.z = 0;
        Instantiate(ballPrefab, world, Quaternion.identity);
    }
}
```

확장 우클릭으로 가장 가까운 공을 삭제하는 기능을 추가해 보라 (Physics2D.OverlapPoint(world)).

🔨 과제 2.2 — 너무 큰 공은 둘로 쪼개기

공의 반지름이 `maxRadius` 이상이면 둘로 분열되도록 하라.

요구 사항

1. 분열 후 두 공의 면적의 합 = 원래 공의 면적 ($r_1^2 + r_2^2 = r_{old}^2$, 간단히 $r_1 = r_2 = r_{old}/\sqrt{2}$).
2. 두 공은 서로 반대 방향으로 같은 속력으로 출발 (운동량 보존).
3. 무한 분열 방지: 분열 직후 짧은 시간(예: 0.5초) 동안 “막 분열한 공”임을 표시하여 그 사이에 다시 분열되지 않게 한다.

힌트 “막 분열한 공” 플래그는 코루틴으로 관리:

```
private bool justSplit = false;

IEnumerator CooldownAfterSplit() {
    justSplit = true;
    yield return new WaitForSeconds(0.5f);
    justSplit = false;
}

void Update() {
    float r = transform.localScale.x * 0.5f;
    if (!justSplit && r > maxRadius) Split();
}
```

설계 고민 분열된 두 공이 서로 즉시 다시 충돌하면? (거리를 약간 띄워 `Instantiate`하면 해결)

🔄 과제 2.3 — 코루틴으로 자동 스폰

`BallSpawner`가 게임 시작 후 2초마다 새 공을 1개씩 자동 추가하도록 하라.

힌트

```
void Start() {
    /* ... initial spawn ... */
    StartCoroutine(SpawnLoop());
}

IEnumerator SpawnLoop() {
    while (true) {
        yield return new WaitForSeconds(2f);
        SpawnOne();
    }
}
```

확장 시간이 지날수록 스폰 간격이 짧아지도록 만들면 난이도 곡선이 생긴다.

🏆 과제 2.4 — HUD 점수 표시

공이 합쳐질 때마다 `score`를 1씩 올리고, 화면 좌측 상단에 “Score: 7” 처럼 표시하라.

요구 사항

- Hierarchy > Create > UI > Text - TextMeshPro로 텍스트 생성 (자동으로 Canvas 추가).
- `GameManager.cs`에 `public static GameManager Instance`로 싱글톤 구현.
- `Ball.Merge()` 안에서 `GameManager.Instance.AddScore(1)` 호출.

힌트 싱글톤 골격:

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    [SerializeField] private TMPro.TextMeshProUGUI scoreText;
    private int score;

    void Awake() {
        if (Instance != null && Instance != this) {
            Destroy(gameObject);
            return;
        }
        Instance = this;
    }

    public void AddScore(int delta) {
        score += delta;
        scoreText.text = "Score: " + score;
    }
}
```

일반 C#과의 비교

일반 C#/GUI에서는 “HUD 그리기”를 매 프레임 `g.DrawString`으로 직접 호출해야 했다. Unity에서는 `scoreText.text = ...` 한 줄이면 끝이다 --- 언제 그릴지는 Unity의 UI 시스템이 알아서 한다.

9.3. Phase 3 --- 새 게임 만들기 (종합 응용)

Rigidbody2D, Collider2D, Prefab, Coroutine, Input을 모두 사용하는 작은 게임을 처음부터 작성한다. 이 단계에서 Unity가 “코드 분량”에서 얼마나 절약되는지를 가장 강하게 체감한다.

📁 과제 3.1 --- 벽돌 깨기 (Block Breaker)

“공 함치기” 예제의 `BallPrefab`과 `Bouncy` 머티리얼을 그대로 활용하여 벽돌 깨기 게임을 만들어라.

Hierarchy 설계

Walls	(Top, Left, Right - 3면만 막음)
Bricks	(BrickPrefab을 격자로 배치, 예: 8 x 4)
Paddle	(BoxCollider2D, Rigidbody2D Kinematic)
Ball	(Rigidbody2D + CircleCollider2D + Bouncy, 1개)
GameManager	(점수, 게임 오버 표시)

필수 스크립트

- `Paddle.cs` --- 좌우 화살표(또는 A/D)로 패들 이동. `transform.position`을 직접 변경하거나 `rb.velocity.x = h * speed;`를 사용.
- `Brick.cs` --- `OnCollisionEnter2D`에서 `Destroy(gameObject)` + 점수 증가.
- `Ball.cs` --- 게임 시작 시 위쪽으로 발사. 화면 아래로 떨어지면(`transform.position.y < -5`) “Game Over”.
- `GameManager.cs` --- 모든 Brick이 사라지면 “Stage Clear” (`FindObjectsOfType<Brick>().Length == 0`).

요구 사항

1. 공이 벽돌과 부딪히면 벽돌이 **한 번에** 사라져야 한다 (양쪽이 동시에 부르는 버그 주의).
2. 패들에 부딪힐 때, 부딪힌 위치에 따라 반사 각도가 달라지면 가산점. (벡터를 직접 `rb.velocity`로 설정하면 됨.)
3. HUD에 점수와 남은 벽돌 수를 표시한다.

일반 C#과의 비교 (필수, README에 작성)

- 일반 C#/GUI 방식으로 “공의 벽 반사”와 “벽돌과의 충돌 검사”에 사용했어야 할 코드 줄 수와 본 과제에서 우리가 작성한 줄 수를 세어 비교하라.
- 사라진 코드는 어디로 갔는지 (어떤 컴포넌트가 대신 처리하는지) 한 줄씩 짚지어 설명하라.

🔧 과제 3.2 — 중력 토글 시뮬레이터

“공 합치기” 예제에 중력을 도입하고, SPACE 키로 중력 방향을 토글할 수 있게 하라.

요구 사항

- BallPrefab의 `Rigidbody2D.gravityScale = 1`.
- `GameManager.Update`에서 SPACE 입력 시 `Physics2D.gravity = -Physics2D.gravity`; 토글.
- 합치기 로직은 그대로 유지.

확장 토글 시 모든 공의 색을 일순간 반짝이게 만들어 “중력 방향이 바뀌었음”을 시각적으로 알려라 (코루틴 + `SpriteRenderer.color`).

9.4. Phase 4 --- 자유 창작**💡 과제 4 — 자신만의 미니 게임**

지금까지 배운 Unity 개념을 **최소 5개 이상** 사용하여, 자유 주제로 미니 게임을 한 편 완성 하라.

필수 사용 개념 (다음 중 5개 이상 명시적으로 사용)

- `[SerializeField]`로 Inspector 노출 변수
- Awake에서 GetComponent 캐싱
- `Rigidbody2D` 또는 `Collider2D`
- Prefab + Instantiate
- `OnCollisionEnter2D` 또는 `OnTriggerEnter2D`
- Coroutine (IEnumerator, yield return)
- `Time.deltaTime`으로 프레임 무관 이동
- TextMeshProUGUI로 HUD

주제 예시 (자유롭게 변경 가능)

- **Catch the Star** --- 위에서 별이 떨어지고 플레이어가 받는 게임

- **Mini Shooter** --- 위쪽 적이 내려오고 아래 플레이어가 발사
- **Dodge** --- 떨어지는 공을 좌우로 피하는 게임
- **Top-Down Maze** --- 미로 안에서 골인 지점 찾기
- **Flappy 스타일** --- 클릭으로 점프하며 장애물 피하기

평가 기준

1. 동작 --- 에러 없이 Play 가능, 1분 이상 플레이 가능.
2. **Unity 활용** --- 위 개념 중 5개 이상을 실제로 사용 (README에 사용 위치 명시).
3. **비교 글쓰기** --- “이 게임을 일반 C#/GUI 방식 (Timer + Paint 이벤트)으로 짰다면 어디가 더 길어질지” 한 단락으로 README에 기술.
4. **창의성** --- 위 예시와 다른 주제 또는 메커니즘 한 가지 이상 추가 시 가산점.

팁

- 무리하게 큰 게임을 시작하지 말고, “한 가지 메커니즘”만 잘 동작하게 만든 후 확장하라.
- Phase 1~3에서 만든 자산(Prefab, 머티리얼, 사운드)을 그대로 가져다 써도 좋다.

10. 이번 강의에서 배운 핵심 개념

개념

MonoBehaviour 상속
 생명 주기 메서드
 컴포넌트 모델
 GetComponent 캐싱
 Time.deltaTime
 Inspector 직렬화
 new 금지
 가짜 null
 Coroutine

Unity가 인식하는 스크립트
 Awake/Start/Update/FixedUpdate
 상속이 아닌 조합으로 G
 Update에서 매번 호출하거
 프레임 속도
 publi
 MonoBehaviour는 AddCom
 파괴된 Unity 오브젝트는 =
 시간 흐름이

☞ 마무리 도전 과제

1. Unity Editor에서 빈 GameObject를 만들고, LifecycleDemo 스크립트를 붙여 **Console** 창에서 Awake → Start 순서를 직접 확인하라.
2. Mover 스크립트를 작성하여 speed를 Inspector에서 조정하며 오브젝트가 다른 속도로 움직이는 것을 확인하라.
3. “화살표 키로 움직이는 캐릭터”를 Unity에서 작성해 보라. Player.cs의 키 입력 + transform.position 변경으로 충분하다.
4. Bomb 코루틴을 변형하여, 1초 간격으로 Debug.Log를 5번 출력한 후 오브젝트가 스스로 사라지도록 만들라.

Part II

Unity로 체험하는 C#의 핵심 기능들

11. 강의 개요

11.1. 이 강의의 주인공은 “C#”이다

이 과목은 **Unity를 이용한 2D 게임프로그래밍** 강의다. 그동안 우리는 Unity 위에서 2D 게임을 만들어 왔다. 이번 강의에서는 게임을 만드는 것이 아니라, 게임 코드의 뼈대가 되는 **C# 기능 자체를 Unity라는 무대 위에서 직관적으로 체험해 본다.**

- 콘솔: `Console.WriteLine("HP: " + hp);` → 글자만 보인다.
- Unity: `cube.color = Color.red;` → 큐브가 실제로 빨개진다.

같은 `int`, 같은 `enum`, 같은 `event`라도, 화면 위 객체의 색이 바뀌고 위치가 움직이는 모습으로 보면 머리에 더 잘 새겨진다. 이번 강의는 **C#의 핵심 기능 6가지를 Unity라는 무대 위에서 작은 실험으로 직접 만져 보는 시간이다.**

11.2. 학습 목표

- C#의 `enum`, 프로퍼티, `interface`, `event/delegate`, 제네릭/LINQ, `Coroutine`/람다 --- 이 6가지 기능이 어떤 문제를 해결하기 위해 존재하는지 이해한다.
- 각 기능을 콘솔이 아닌 **Unity 화면**에서 직접 동작시켜 본다.
- 기능마다 작은 **Lab(실험실)** 스크립트 하나를 작성하고, 키 입력으로 동작을 체험한다.
- 게임을 “완성”하는 것이 목표가 아니라, C# 기능 하나하나를 **확실히 손에 익히는 것**이 목표다.

11.3. 왜 “Lab(실험실)” 형식인가?

게임을 한 편 만드는 것도 좋지만, 그러면 “이 코드 한 줄이 정말로 무슨 일을 하는가”가 게임 전체의 흐름에 묻힌다. 이번 강의는 그 반대다. 큰 게임을 만들지 않는다. 대신 **6개의 작은 실험**을 만든다.

Lab의 형식

1. 문제: “이런 상황이라면 어떻게 코드를 써야 할까?”
2. 콘솔 비교: 일반 C# 콘솔 프로그램이라면 어떻게 했을까?
3. Unity 데모: 작은 `GameObject` + 짧은 스크립트.
4. 키 입력: 1, 2, H, D, A, SPACE 등으로 동작 확인.
5. 실험해 보기: 코드의 한 줄을 바꾸면 결과가 어떻게 달라지는지 손으로 확인.

11.4. 이전 강의들과의 연결

강의	무엇을 배웠나	이번 강의에서의 역할
Lec 1--8	Unity 2D 게임 개발 경험	이번 Lab에서 “쓰는 도구”
Lec 9	Unity C# 스크립트 기초	<code>MonoBehaviour</code> , <code>Update</code> 는 안다고 가정
Lec 10	C# 핵심 기능 6가지를 시각적 실험으로	언어 기능을 손에 익히기

12. 프로젝트 준비

12.1. Unity 프로젝트 만들기

Unity Hub에서 **New Project** → **3D (Built-in Render Pipeline)** 또는 **2D** 어느 쪽도 무방하다 (이번 강의는 3D 큐브를 가정). 프로젝트 이름은 CSharpFeatureLab.

Lab 공통 셋업

썬에 다음 GameObject들을 만들어 둔다 (각 Lab에서 재사용).

- Main Camera --- 기본값 그대로.
- Directional Light --- 기본값 그대로 (3D 모드라면).
- LabCube --- 3D Object → Cube. 위치 (0, 0, 0). Lab 1에서 “무드 큐브”로 사용한다.
- LabRoot --- 빈 GameObject. 우리가 작성할 Lab 스크립트들을 여기에 붙인다.
- Canvas + LogText --- UI Canvas 안에 TextMeshPro - Text 하나. LogText라 이름 두고 화면 좌상단에 배치. 결과를 화면에서도 확인하기 위함.

콘솔 창(Window → General → Console)을 열어 두고 Debug.Log 결과를 동시에 본다.

12.2. 공통 도우미 스크립트: LogPanel.cs (선택)

Debug.Log 만 써도 충분하지만, 화면에서 결과를 직접 확인하고 싶다면 다음 스크립트를 Canvas에 붙인다.

```

1 using TMPro;
2 using UnityEngine;
3
4 public class LogPanel : MonoBehaviour
5 {
6     public static LogPanel Instance { get; private set; }
7     [SerializeField] private TMP_Text text;
8
9     private string buffer = "";
10
11     void Awake() { Instance = this; }
12
13     public void Print(string msg)
14     {
15         Debug.Log(msg);           // Console
16         buffer = msg + "\n" + buffer; // newest on top
17         if (buffer.Length > 800)
18             buffer = buffer.Substring(0, 800);
19         if (text != null) text.text = buffer;
20     }
21 }
```

이후 Lab 코드에서는 Debug.Log(...) 대신 LogPanel.Instance.Print(...)을 사용해도 된다. 이 강의에서는 둘 다 허용한다.

13. Lab 1 --- enum과 switch 표현식

13.1. 문제

세 가지 “기분(mood)” 상태 중 하나를 표현하고 싶다. int 0/1/2로 쓰면 “state == 1이 무슨 뜻이지?”라는 혼란이 생기고, 오타도 잡히지 않는다. 이름 있는 상수 집합이 필요하다.

13.2. 콘솔에서라면

일반 C# 콘솔 프로그램이었다면 이렇게 했을 것이다.

```

1 public enum Mood { Happy, Sad, Angry }
2
3 class Program
4 {
5     static void Main()
6     {
7         Mood m = Mood.Happy;
8         string msg = m switch
9         {
10             Mood.Happy => "I am happy",
11             Mood.Sad    => "I am sad",
12             Mood.Angry  => "I am angry",
13             -           => "??"
14         };
15         Console.WriteLine(msg);
16     }
17 }
```

콘솔이라면 “I am happy”라는 글자가 한 번 출력되고 끝이다.

13.3. Unity 데모

큐브 하나를 두고, 키 1/2/3을 누르면 Mood가 바뀌면서 큐브의 색이 즉시 바뀐다.

```

1 using UnityEngine;
2
3 public enum Mood { Happy, Sad, Angry }
4
5 public class Lab1_Enum : MonoBehaviour
6 {
7     [SerializeField] private Renderer targetRenderer;
8     private Mood currentMood = Mood.Happy;
9
10    void Start()
11    {
12        Apply(currentMood);
13    }
14
15    void Update()
16    {
17        if (Input.GetKeyDown(KeyCode.Alpha1)) Apply(Mood.Happy);
18        if (Input.GetKeyDown(KeyCode.Alpha2)) Apply(Mood.Sad);
19        if (Input.GetKeyDown(KeyCode.Alpha3)) Apply(Mood.Angry);
20    }
21
22    void Apply(Mood m)
23    {
24        currentMood = m;
25
26        // C# 8+ switch expression
```

```

27     Color color = m switch
28     {
29         Mood.Happy => Color.yellow,
30         Mood.Sad   => new Color(0.3f, 0.5f, 1f),
31         Mood.Angry => Color.red,
32         -          => Color.white
33     };
34
35     targetRenderer.material.color = color;
36     Debug.Log($"Mood = {m}");
37 }
38 }

```

13.4. Unity 구현 가이드

1. **씬에 큐브 만들기**
Hierarchy 창에서 우클릭 → 3D Object → Cube. 새로 생긴 객체의 이름을 MoodCube로 바꾼다. Inspector에서 Transform.Position을 (0, 0, 0)으로 둔다.
2. **Lab1 GameObject 만들기**
Hierarchy 창에서 우클릭 → Create Empty. 이름을 Lab1로 바꾼다. 이 빈 객체에 우리 스크립트를 붙일 것이다.
3. **스크립트 작성**
Project 창에서 Assets/Scripts/ 폴더 우클릭 → Create → C# Script. 이름을 Lab1_Enum으로 만든다. 더블 클릭해 VS Code에서 열고, 위의 Lab1_Enum.cs 코드를 복사해 넣는다.
4. **스크립트 부착**
Project 창의 Lab1_Enum 스크립트를 드래그해 Hierarchy의 Lab1 객체 위에 놓는다. Lab1을 선택하면 Inspector에 Lab1_Enum (Script) 컴포넌트가 보인다.
5. **Inspector 슬롯 연결**
Target Renderer라는 슬롯이 비어 있다 (None (Renderer)). Hierarchy의 MoodCube를 드래그해 이 슬롯에 떨어뜨린다.
6. **재생**
화면 상단의 Play 버튼을 누른다. 키 1, 2, 3을 누를 때마다 큐브가 노란/파란/빨간색으로 바뀐다. Console 창(Window → General → Console)에 Mood = ...가 함께 출력된다.

드래그한 것은 GameObject인데, 왜 Renderer 슬롯에 들어가나?

Inspector 슬롯의 타입이 Renderer일 때 GameObject를 드래그하면, Unity가 그 **GameObject**에 붙어 있는 컴포넌트들 중 타입이 일치하는 것을 자동으로 찾아 슬롯에 채운다. MoodCube에는 MeshRenderer(=Renderer의 자식 클래스)가 있으므로 그게 들어간다. 이는 Unity Editor의 편의 기능이며, 코드로는 moodCube.GetComponent<Renderer>()와 같다.

13.5. 여기서 배우는 C# 개념

enum: 이름 있는 상수의 집합

enum Mood { Happy, Sad, Angry }는 내부적으로 0, 1, 2라는 int일 뿐이다. 하지만 코드에 `state == 1`이라고 쓰는 것과 `state == Mood.Sad`라고 쓰는 것은 읽기 쉬움이 천지차이다. 오타는 컴파일러가 잡아 주고, IDE는 자동 완성을 도와준다.

switch 표현식 (C# 8.0+)

전통적인 switch는 문(statement)이라 break와 case를 일일이 써야 했다. C# 8부터 switch는 식(expression)이 되어 값을 반환한다.

```

1 // Old style (statement)
2 Color color;
3 switch (m)
4 {
5     case Mood.Happy: color = Color.yellow; break;
6     case Mood.Sad:   color = Color.blue;   break;
7     default:        color = Color.white;  break;
8 }
9
10 // New style (expression)
11 Color color = m switch
12 {
13     Mood.Happy => Color.yellow,
14     Mood.Sad   => Color.blue,
15     _          => Color.white
16 };

```

짧고, “무엇을 얻는가”가 한눈에 보인다. _는 “나머지 모든 경우”.

[SerializeField] private이 무슨 뜻인가?

private 필드는 같은 클래스 안에서만 접근 가능하다 (캡슐화). 하지만 그렇게만 두면 Unity Inspector에서도 보이지 않아 값을 조정할 수 없다. [SerializeField]는 “이 필드를 Inspector에 보여 줘”라고 Unity에 알려 주는 어트리뷰트(Attribute)다.

결과적으로 [SerializeField] private float speed = 8f;는 다음을 의미한다:

- 다른 C# 클래스에서는 lab.speed로 접근 불가.
- Unity Editor의 Inspector에서는 값을 직접 수정 가능.

대괄호 안의 [Foo] 같은 표기는 모두 어트리뷰트이며, “클래스/필드/메서드에 꼬리표를 붙인다”고 이해하면 된다.

\$"...{x}..." --- 문자열 보간(string interpolation)

큰따옴표 앞에 \$를 붙이면 문자열 안의 {식}이 그 식의 값으로 치환된다.

```

1 int n = 42;
2 string s = $"n = {n}";           // "n = 42"
3 string t = $"sum = {a + b}";     // expressions also work
4 string u = $"avg = {x:F2}";      // F2 = two decimal places

```

콜론 뒤의 “F2”, “D6” 등은 형식 지정자. F2는 “소수점 둘째 자리까지”, D6은 “정수 6자리, 빈 자리는 0”.

옛 방식의 `string.Format("n = {0}", n)`이나 `"n = " + n`보다 훨씬 읽기 쉽다.

실험해 보기

- Mood에 Excited를 추가하고, 키 4에 연결해 보자. switch의 _ 가지가 어떤 역할인지 확인.
- Mood.Happy를 정수로 바꾸려면? `(int)Mood.Happy`를 `Debug.Log`로 출력해 보자.
- `enum Mood : byte`로 하면 어떻게 달라질까? (저장 크기 변화)

14. Lab 2 --- 프로퍼티와 캡슐화

14.1. 문제

캐릭터의 HP는 외부에서 읽는 것은 OK, 마음대로 쓰는 것은 안됨이어야 한다. 누구나 `character.hp = 99999`로 만들 수 있으면 게임 규칙이 무너진다.

14.2. 콘솔에서라면

```

1 class Character
2 {
3     public int hp;    // anyone can write to this - dangerous
4 }
5
6 class Program
7 {
8     static void Main()
9     {
10         var c = new Character();
11         c.hp = 100;
12         c.hp = -50;    // negative? no way to block this
13     }
14 }
```

이 코드는 캡슐화 위반이다. 프로퍼티로 다음과 같이 보호한다.

```

1 class Character
2 {
3     public int Hp { get; private set; }    // public read, blocked write
4     private int maxHp = 10;
5
6     public Character() { Hp = maxHp; }
7
8     public void Damage(int amount)
9     {
10         Hp = Math.Max(0, Hp - amount);
11     }
12
13     public void Heal(int amount)
14     {
15         Hp = Math.Min(maxHp, Hp + amount);
16     }
17 }
```

14.3. Unity 데모

같은 Character 로직을 키 입력에 연결한다. D 키로 데미지, H 키로 회복, R 키로 “밖에서 직접 쓰기 시도”(컴파일 에러).

```

1 using UnityEngine;
2
3 public class Lab2_Property : MonoBehaviour
4 {
5     [SerializeField] private int maxHp = 10;
6     public int Hp { get; private set; }
7
8     void Start()
9     {
10         Hp = maxHp;
11         Report();
12     }
13
14     void Update()
15     {
16         if (Input.GetKeyDown(KeyCode.D)) { Damage(1); Report(); }
17         if (Input.GetKeyDown(KeyCode.H)) { Heal(1); Report(); }
18
19         // The line below would NOT compile, by design:
20         // if (Input.GetKeyDown(KeyCode.R)) Hp = 9999;
21     }
22
23     public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
24     public void Heal(int amount) => Hp = Mathf.Min(maxHp, Hp + amount);
25
26     void Report()
27     {
28         Debug.Log($"HP = {Hp}/{maxHp}");
29     }
30 }

```

14.4. Unity 구현 가이드

1. Lab2 GameObject 만들기

Hierarchy에서 우클릭 → Create Empty → 이름 Lab2. 이번 Lab은 시각적 객체가 필요 없다 (Console로만 결과 확인).

2. 스크립트 작성 & 부착

Assets/Scripts/Lab2_Property.cs를 만들어 코드 입력 후 Lab2 객체에 드래그.

3. Inspector에서 값 확인

Lab2를 선택해 Inspector를 보면 Max Hp = 10이 보인다. 이는 코드의 [SerializeField] private int maxHp = 10; 덕분. Hp는 보이지 않는다 --- [SerializeField]가 없는 그냥 자동 프로퍼티이기 때문. 값을 20으로 바꾸면 시작 HP가 20/20이 된다.

4. Console 창 열기

Window → General → Console. Play 직전에 Console을 봐 두면 Debug.Log 출력이 즉시 보인다.

5. 재생

Play → D 키로 데미지, H 키로 회복. Console에 HP = 9/10, HP = 8/10 같은 로그가 누적된

다. Hp가 0 미만 또는 maxHp 초과로 가지 않는다 (Mathf.Max/Mathf.Min 덕분).

14.5. 여기서 배우는 C# 개념

프로퍼티: { get; private set; }

필드(field)는 “값을 담는 변수”. 프로퍼티(property)는 “필드처럼 보이지만 실은 메서드 두 개”다.

- public int Hp { get; set; } --- 외부 읽기 OK, 외부 쓰기 OK (필드와 거의 같음).
- public int Hp { get; private set; } --- 외부 읽기 OK, 외부 쓰기 차단.
- public int Hp { get; } --- 읽기 전용 (생성자에서만 초기화 가능).

이 한 줄로 캡슐화의 의도가 분명히 드러난다. “Hp는 외부에서 보는 값이다”.

식 본문 메서드: =>

Damage(int amount) { Hp = ...; }처럼 한 줄짜리 메서드는 =>로 줄일 수 있다.

```
1 public void Damage(int amount) => Hp = Mathf.Max(0, Hp - amount);
```

읽기 좋고 짧다. 본문이 두 줄 이상이면 사용하지 않는다.

{ get; private set; }을 풀어 보면?

public int Hp { get; private set; }는 자동 구현 프로퍼티(auto-implemented property)다. 컴파일러가 다음과 같은 코드를 자동으로 만들어 준다.

```
1 private int _hp;           // hidden backing field generated by the compiler
2 public int Hp
3 {
4     get { return _hp; }     // public read for anyone
5     private set { _hp = value; } // write only inside this class
6 }
```

value는 set이 받은 값을 가리키는 약속된 키워드. Hp = 10; 이라고 쓰면 set의 value에 10이 들어간다.

읽기/쓰기 옵션을 다양하게 줄 수 있다:

- { get; set; } --- 외부 읽기 OK, 외부 쓰기 OK
- { get; private set; } --- 외부 읽기 OK, 외부 쓰기 차단
- { get; } --- 읽기만, 쓰기 불가능 (생성자에서만 초기화)

=> 화살표의 세 가지 얼굴

초보자에게 헷갈리는 점: C#의 =>는 문맥에 따라 세 가지로 쓰인다. 모두 “왼쪽이 들어가서 오른쪽이 나온다”는 비슷한 느낌이지만, 자리와 의미가 다르다.

(1) 식 본문 멤버 (Lab 2, Lab 3에서 만남) “이 메서드/프로퍼티는 이 식 한 줄을 반환한다”.

```
1 public void Damage(int n) => Hp -= n;           // method
2 public string Name => "Circle";                 // property
```

```
3 public float Area() => Mathf.PI * r * r;    // method
```

같은 자리에 { ... }를 쓸 수도 있다. =>는 단축 표기일 뿐.

(2) 람다 식 (Lab 5, Lab 6에서 만남) “이 매개변수들을 받아서 이 식을 반환하는 함수”.

```
1 x => x >= 80                                // takes int, returns bool (Func<int, bool>)
2 n => Debug.Log($"Tick {n}")                 // takes int, returns void (Action<int>)
3 () => Debug.Log("hi")                       // no parameters (Action)
4 (a, b) => a + b                             // two parameters, returns sum (Func<int,int,int>)
```

이는 이름 없는 함수이며, 변수에 담거나 메서드 인자로 넘길 수 있다.

(3) switch 식의 가지 (Lab 1에서 만남) “이 패턴이면 이 값”.

```
1 Color c = m switch {
2     Mood.Happy => Color.yellow,    // when Happy, return yellow
3     Mood.Sad   => Color.blue,
4     -          => Color.white
5 };
```

기억하는 법: 어디에 있느냐를 보면 된다. 메서드/프로퍼티 선언 자리면 (1), 인자/변수 자리에서 함수 자체를 만드는 거면 (2), switch 안이면 (3).

실험해 보기

- 위 코드의 `public int Hp { get; private set; }`을 `public int Hp;`으로 바꿔 보자. 그러면 다른 스크립트(예: Lab2_Bad.cs)에서 `lab.Hp = 9999`로 마음대로 바꿀 수 있다. “프로퍼티”의 의의가 체감된다.
- `Hp { get; private set; }`을 `Hp { get; }`으로 바꿔 보자. 이제 Damage 안의 `Hp = ...`도 컴파일 에러가 난다. 어디까지 “쓰기”를 허용할 것인가를 한 줄로 조절하는 감각을 익힌다.

15. Lab 3 --- 인터페이스 (interface)

15.1. 문제

여러 도형(Circle, Rectangle, Triangle)의 면적을 구하고 출력하고 싶다. 도형마다 면적을 구하는 공식은 다르지만, “면적을 구한다”라는 능력은 공통이다. 이 “공통 능력”만 약속해 두면, 도형이 늘어나도 호출 코드는 바뀌지 않는다.

15.2. 콘솔에서라면

```
1 public interface IShape
2 {
3     string Name { get; }
4     float Area();
5 }
6
7 public class Circle : IShape
8 {
9     public float Radius;
10    public Circle(float r) { Radius = r; }
11    public string Name => "Circle";
12    public float Area() => Mathf.PI * Radius * Radius;
```

```

13 }
14
15 public class Rectangle : IShape
16 {
17     public float W, H;
18     public Rectangle(float w, float h) { W = w; H = h; }
19     public string Name => "Rectangle";
20     public float Area() => W * H;
21 }
22
23 class Program
24 {
25     static void Main()
26     {
27         IShape[] shapes = { new Circle(2f), new Rectangle(3f, 4f) };
28         foreach (var s in shapes)
29             Console.WriteLine($"{s.Name}: {s.Area():F2}");
30     }
31 }

```

15.3. Unity 데모

같은 IShape를 Unity에서 사용한다. A 키를 누르면 모든 도형의 면적이 콘솔/화면에 출력된다.

```

1 using UnityEngine;
2
3 public interface IShape
4 {
5     string Name { get; }
6     float Area();
7 }
8
9 public class CircleShape : IShape
10 {
11     private float radius;
12     public CircleShape(float r) { radius = r; }
13     public string Name => "Circle";
14     public float Area() => Mathf.PI * radius * radius;
15 }
16
17 public class RectangleShape : IShape
18 {
19     private float w, h;
20     public RectangleShape(float w, float h) { this.w = w; this.h = h; }
21     public string Name => "Rectangle";
22     public float Area() => w * h;
23 }
24
25 public class Lab3_Interface : MonoBehaviour
26 {
27     private IShape[] shapes;
28
29     void Start()
30     {
31         shapes = new IShape[]
32         {
33             new CircleShape(2f),
34             new RectangleShape(3f, 4f),

```

```

35         new CircleShape(1.5f),
36     };
37 }
38
39 void Update()
40 {
41     if (Input.GetKeyDown(KeyCode.A))
42     {
43         float total = 0f;
44         foreach (var s in shapes)
45         {
46             Debug.Log($"{s.Name}: area = {s.Area():F2}");
47             total += s.Area();
48         }
49         Debug.Log($"Total area = {total:F2}");
50     }
51 }
52 }

```

15.4. Unity 구현 가이드

1. Lab3 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab3.

2. 스크립트 파일 정리

이번 Lab에는 클래스가 여러 개 등장한다 (IShape, CircleShape, RectangleShape, Lab3_Interface). 한 .cs 파일에 모두 넣어도 되지만, 권장 구조는 한 파일 = 한 클래스. Assets/Scripts/ 안에 다음 네 파일을 만든다:

- IShape.cs --- 인터페이스 정의
- CircleShape.cs --- 원
- RectangleShape.cs --- 사각형
- Lab3_Interface.cs --- MonoBehaviour

3. 스크립트 부착

Lab3_Interface만 Lab3 GameObject에 드래그. IShape, CircleShape, RectangleShape는 붙이지 않는다. 이들은 MonoBehaviour가 아니라 보통 C# 클래스이며, Lab3_Interface.Start() 안에서 new로 만들어진다.

4. Console 창 열기

Window → General → Console.

5. 재생

Play → A 키. Console에 다음과 같은 로그가 출력된다:

```

Circle: area = 12.57
Rectangle: area = 12.00
Circle: area = 7.07
Total area = 31.63

```

Unity의 “파일명 = 클래스명” 규칙

MonoBehaviour를 상속한 클래스는 파일 이름과 클래스 이름이 반드시 같아야 Unity가 컴포넌트로 인식한다. 하지만 보통 C# 클래스나 인터페이스는 그 규칙이 없다. 한 파일에 여러 개의 비-MonoBehaviour 클래스를 뒤도 동작하지만, 읽기 좋고 찾기 쉽도록 “한 파일 한 클래스”를 권한다 (Lec 9에서도 다룬 규칙).

15.5. 여기서 배우는 C# 개념

interface: “무엇을 할 수 있는가” 약속

인터페이스는 메서드/프로퍼티의 목록만 정의한다. 구현은 각 클래스가 자유롭게 한다.

- 클래스는 여러 인터페이스를 동시에 구현할 수 있다 (다중 “상속”의 안전한 형태).
- 인터페이스 자체는 **new**로 만들 수 없다 --- 구현체가 있어야.
- 이름은 보통 **I**로 시작 (IShape, IComparable, IEnumerable...).

상속(is-a) vs 인터페이스(can-do) 상속은 “Cat은 Animal이다”라는 is-a 관계.

인터페이스는 “Cat은 울 수 있다”라는 can-do 관계.

“면적을 구할 수 있다”는 후자에 해당하므로 인터페이스가 적합하다.

프로퍼티가 인터페이스에 들어갈 수 있다

이번 IShape는 Name이라는 프로퍼티도 약속에 포함시켰다. 인터페이스는 메서드뿐 아니라 프로퍼티/이벤트/인덱서도 약속할 수 있다.

public string Name => "Circle";이 무슨 뜻인가?

식 본문 프로퍼티(expression-bodied property) --- “이 프로퍼티는 항상 이 식의 값을 반환한다”.

풀어 쓰면 다음과 같다:

```
1 // Short form
2 public string Name => "Circle";
3
4 // Same meaning, expanded
5 public string Name
6 {
7     get { return "Circle"; }
8 }
```

세 가지 사실:

- set이 없으므로 외부에서 쓸 수 없다 (circle.Name = "X" → 컴파일 에러).
- 호출할 때마다 식의 값을 매번 평가해서 반환한다 (이번 경우 항상 "Circle").
- 메서드(Name())가 아니라 프로퍼티이므로 사용 시 괄호 없이 circle.Name이라고 쓴다.

이는 “=>의 세 얼굴” 중 (1) 식 본문 멤버에 해당한다 (Lab 2 박스 참고).

foreach와 var

foreach (var s in shapes)는 “shapes의 원소를 하나씩 꺼내 s에 담아 본다”라는 뜻.

var: 타입 추론 var s는 “타입을 컴파일러가 알아서 결정해 주세요”라는 표시. shapes가 IShape[]이므로 s의 타입은 IShape로 자동 결정된다.

주의: var는 “아무 타입이나” 의미가 아니다. 타입은 컴파일 시점에 결정되며, 한 번 정해지면 바꿀 수 없다. JavaScript의 var나 Python의 동적 타입과는 전혀 다르다.

실험해 보기

- TriangleShape를 추가하고 shapes 배열에 끼워 넣자.
Lab3_Interface.cs의 Update는 한 줄도 고치지 않아도 새 도형이 같이 출력된다. “인터페이스의 효과”를 손으로 느낀다.
- IShape에 Perimeter()를 추가해 보자. 그러면 모든 구현 클래스가 컴파일 에러를 낸다.
약속을 안 지키면 컴파일러가 막아 준다는 점이 중요하다.

16. Lab 4 --- delegate, Action, event

16.1. 문제

“숫자 변수가 바뀌면 여러 곳에 알리고 싶다.” UI 텍스트도 갱신되어야 하고, 사운드도 재생되어야 하고, 화면 흔들림도 일어나야 한다. 이를 변수 쪽에서 직접 호출하면, 알림 받는 쪽이 새로 추가될 때마다 코드가 늘어난다.

16.2. 콘솔에서라면

이벤트로 “소문”을 내고, 관심 있는 사람만 듣는 구조를 만들 수 있다.

```

1 class Counter
2 {
3     public event Action<int> OnChanged;    // notification only
4     private int n = 0;
5     public void Increment()
6     {
7         n++;
8         OnChanged?.Invoke(n);
9     }
10 }
11
12 class Program
13 {
14     static void Main()
15     {
16         var c = new Counter();
17         c.OnChanged += v => Console.WriteLine($"Logger: {v}");
18         c.OnChanged += v => Console.WriteLine($"Beeper: beep {v} times!");
19
20         c.Increment();    // triggers BOTH subscribers
21         c.Increment();
22     }
23 }
```


16.3. Unity 데모

스페이스 키를 누르면 Counter가 1씩 증가하고, 여러 구독자가 자동으로 반응한다.

```

1 using System;
2 using UnityEngine;
3
4 public class Lab4_Counter : MonoBehaviour
5 {
6     public event Action<int> OnChanged;
7     private int value_ = 0;
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.Space))
12        {
13            value_++;
14            OnChanged?.Invoke(value_);
15        }
16    }
17 }

```

구독자(observer)는 별도 컴포넌트로 만든다.

```

1 using UnityEngine;
2
3 public class Lab4_Logger : MonoBehaviour
4 {
5     [SerializeField] private Lab4_Counter source;
6
7     void OnEnable() { source.OnChanged += React; }
8     void OnDisable() { source.OnChanged -= React; }
9
10    void React(int v) => Debug.Log($"Logger heard: {v}");
11 }
12
13 public class Lab4_ColorChanger : MonoBehaviour
14 {
15     [SerializeField] private Lab4_Counter source;
16     [SerializeField] private Renderer target;
17
18     void OnEnable() { source.OnChanged += React; }
19     void OnDisable() { source.OnChanged -= React; }
20
21     void React(int v)
22     {
23         // 0 -> dark, 10 -> bright, then loops
24         float t = (v % 10) / 10f;
25         target.material.color = Color.Lerp(Color.black, Color.cyan, t);
26     }
27 }

```

16.4. Unity 구현 가이드

이번 Lab은 여러 컴포넌트가 서로 참조하므로, 단계가 조금 길다. 차분히 따라가자.

1. 발신자 **GameObject** 만들기

Hierarchy → Create Empty → 이름 Lab4_Source. Lab4_Counter.cs를 작성하여 이 객체에 드래그.

2. 큐브 만들기

Hierarchy → 3D Object → Cube → 이름 Lab4_Cube. 위치 (2, 0, 0) 정도로 두면 다른 Lab의 큐브와 안 겹친다.

3. 구독자 **GameObject** 만들기

Hierarchy → Create Empty → 이름 Lab4_Subscribers. 이 객체에 두 개의 스크립트를 모두 드래그:

- Lab4_Logger.cs
- Lab4_ColorChanger.cs

한 GameObject에 여러 컴포넌트가 붙는다. 이는 Unity에서 흔한 패턴이다.

4. **Inspector** 슬롯 연결 (가장 중요한 단계)

Lab4_Subscribers를 선택. Inspector에 두 컴포넌트가 모두 보인다.

- Lab4_Logger의 Source 슬롯 ← Lab4_Source 드래그. Unity가 자동으로 Lab4_Counter 컴포넌트를 찾아 채운다.
- Lab4_ColorChanger의 Source 슬롯 ← Lab4_Source 드래그.
- Lab4_ColorChanger의 Target 슬롯 ← Lab4_Cube 드래그.

5. 재생

Play → SPACE 키. 콘솔에 로그가 찍히면서 동시에 큐브 색이 변한다. 두 구독자는 서로의 존재를 모른다.

GameObject 구조를 그림으로

Hierarchy:

Main Camera

Directional Light

Lab4_Source <-- Lab4_Counter (발신자)

Lab4_Subscribers <-- Lab4_Logger, Lab4_ColorChanger (수신자 둘)

Lab4_Cube <-- Renderer (색이 바뀜)

이렇게 발신자/수신자/시각적 객체를 세 개의 GameObject로 책임 분리하면 구조가 더 명확해진다. 같은 GameObject에 모두 붙여도 동작은 하지만, 학습 단계에서는 분리해 두는 편이 좋다.

16.5. 여기서 배우는 C# 개념

delegate, Action, event 한 묶음

delegate: 메서드를 변수처럼 담는 그릇. `public delegate void InHandler(int x);`처럼 직접 정의할 수도 있지만, 보통은 ...

Action<T>: “T를 받고 void를 반환하는 delegate”를 .NET이 미리 만들어 둔 것.

Func<T, R>: “T를 받고 R을 반환하는 delegate”.

event: delegate 위에 “외부에서는 +=와 -=만 가능”이라는 안전 장치를 씌운 것. 외부에서 `c.OnChanged()`로 직접 호출하지는 못한다.

?.Invoke(...): 안전 호출

event는 처음에 null이다. 아무도 구독하지 않았는데 `Invoke`를 부르면 `NullReferenceException`. `?.(null-conditional 연산자)`을 쓰면 “왼쪽이 null이 아닐 때만”만 호출한다.

`OnChanged?.Invoke(value_);` == `if (OnChanged != null) OnChanged(value_);`

OnDisable에서 반드시 -=

`+=`로 구독한 메서드는 `-=`로 해제하지 않으면 그 객체가 파괴된 후에도 이벤트가 그쪽을 호출하려 한다. 이는 메모리 누수와 파괴된 객체에 호출이라는 두 가지 문제를 동시에 만든다. 구독은 `OnEnable`, 해제는 `OnDisable`이 안전한 짝.

Action<T>, Func<T, R>, delegate의 정체

delegate는 “메서드를 담는 변수의 타입”이다. .NET에는 자주 쓰는 두 가지 모양이 미리 만들어져 있다.

```
1 // "delegate that takes T and returns void"
2 public delegate void Action<T>(T arg);
3
4 // "delegate that takes T and returns R"
5 public delegate R Func<T, R>(T arg);
```

이 둘은 우리가 직접 정의한 게 아니라 .NET이 미리 만들어 놓은 단축이다. 덕분에 우리가 `public delegate void MyHandler(int x);`처럼 매번 새로 정의할 필요가 없다.

- `Action` --- 매개변수 없음, 반환 `void`.
- `Action<int>` --- `int` 받음, 반환 `void`.
- `Action<int, string>` --- `int`와 `string` 받음, 반환 `void`.
- `Func<int, bool>` --- `int` 받음, `bool` 반환.
- `Func<int, int, int>` --- 마지막이 반환, 앞은 매개변수.

?.와 ?.Invoke(...)

?.는 **null-조건 연산자(null-conditional operator)**. “왼쪽이 null이 아닐 때만” 오른쪽을 실행한다.

```
1 string s = null;
2 int len = s.Length;           // NullReferenceException!
3 int? len2 = s?.Length;       // if s is null, len2 is also null. Safe.
```

이벤트와의 결합

event는 처음에 null이다 (아직 아무도 구독하지 않은). 이때 OnChanged()를 직접 호출하면 NullReferenceException. ? Invoke(...)은 객체*구독자가 한 명이라도 있을 때 호출, 없으면 그냥 패스.

```
1 // Equivalent two lines
2 OnChanged?.Invoke(value_);
3 if (OnChanged != null) OnChanged(value_);
```

왜 Invoke를 써야 하나?

event는 외부에서 OnChanged()로 직접 호출이 막혀 있다 (안전 장치). 대신 Invoke()라는 메서드가 있어, “이름과 매개변수 형태가 같은 어떤 메서드든 호출” 가능. Invoke()와 OnChanged?.Invoke(...) 메서드가 사실상 표준.

OnEnable/OnDisable이 호출되는 시점

Lec 9에서 다룬 MonoBehaviour의 생명 주기 메서드 중 두 개:

- **OnEnable** --- 컴포넌트가 활성화될 때마다 호출. Play 시작 직후, 또는 Inspector의 체크박스를 켤 때, 또는 gameObject.SetActive(true) 직후.
- **OnDisable** --- 컴포넌트가 비활성화될 때마다 호출. 체크박스 끄기, SetActive(false), 객체 파괴 직전.

이벤트 구독을 Start에서 하고 해제를 OnDestroy에서 해도 동작은 한다. 하지만 OnEnable/OnDisable을 쓰면 컴포넌트가 켜졌다 꺼졌다 할 때마다 자동으로 짝이 맞아 더 안전하다.

실험해 보기

- Lab4_ColorChanger의 enabled를 끄면, 큐브 색은 변하지 않지만 로그는 여전히 찍힌다. 구독자가 “빠지더라도” 발신자는 영향받지 않는다는 점을 확인한다.
- 새 구독자 Lab4_Beeper.cs를 추가해 본다. 발신자(Lab4_Counter)는 한 줄도 수정하지 않는다. 이것이 “느슨한 결합”의 효과다.

17. Lab 5 --- 제네릭 List<T>와 LINQ 맛보기

17.1. 문제

여러 점수를 모아 두고, “총합/평균/최고점/80점 이상의 개수”를 알고 싶다. 배열은 크기가 고정되어 “계속 추가”가 어렵다. 크기가 자동으로 늘어나는 컬렉션이 필요하다.

17.2. 콘솔에서라면

```
1 using System.Collections.Generic;
2 using System.Linq;
3
4 class Program
5 {
6     static void Main()
7     {
8         List<int> scores = new List<int>();
9         scores.Add(85); scores.Add(72); scores.Add(91); scores.Add(60);
10    }
```

```

11 Console.WriteLine($"Count : {scores.Count}");
12 Console.WriteLine($"Sum : {scores.Sum()}");
13 Console.WriteLine($"Avg : {scores.Average():F1}");
14 Console.WriteLine($"Max : {scores.Max()}");
15 Console.WriteLine($">=80 : {scores.Where(s => s >= 80).Count()}");
16 }
17 }

```

17.3. Unity 데모

N 키를 누르면 0~99 범위 랜덤 점수를 List에 추가하고, L 키를 누르면 LINQ로 통계를 출력한다.

```

1 using System.Collections.Generic;
2 using System.Linq;
3 using UnityEngine;
4
5 public class Lab5_Generic : MonoBehaviour
6 {
7     private List<int> scores = new List<int>();
8
9     void Update()
10    {
11        if (Input.GetKeyDown(KeyCode.N))
12        {
13            int s = Random.Range(0, 100);
14            scores.Add(s);
15            Debug.Log($"Added: {s}, Count={scores.Count}");
16        }
17
18        if (Input.GetKeyDown(KeyCode.L))
19        {
20            if (scores.Count == 0)
21            {
22                Debug.Log("No scores yet. Press N to add.");
23                return;
24            }
25
26            int sum = scores.Sum();
27            double avg = scores.Average();
28            int max = scores.Max();
29            int top = scores.Where(x => x >= 80).Count();
30
31            Debug.Log($"Sum={sum}, Avg={avg:F1}, Max={max}, >=80: {top}");
32        }
33
34        if (Input.GetKeyDown(KeyCode.K))
35        {
36            scores.Clear();
37            Debug.Log("Cleared.");
38        }
39    }
40 }

```

17.4. Unity 구현 가이드

1. Lab5 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab5.

2. 스크립트 작성 & 부착

Assets/Scripts/Lab5_Generic.cs를 만들어 코드 입력. Lab5 객체에 드래그해 부착.

3. using 지시문 확인

파일 맨 위에 다음 두 줄이 반드시 있어야 한다:

```
using System.Collections.Generic; // List<T>를 위해
using System.Linq;                // .Sum(), .Where() 등을 위해
```

둘 중 하나라도 빠지면 List나 .Sum()이 “컴파일 에러”로 표시된다. VS Code에서 빨간 줄이 보이면 using부터 점검.

4. Console 창 열기

Window → General → Console.

5. 재생

Play 후 다음 키를 눌러 본다:

- **N** --- 점수를 하나 추가 (0~99 랜덤)
- **L** --- 통계 출력 (Sum, Avg, ≥80 개수)
- **K** --- 리스트 비우기

N을 여러 번 누른 후 L을 눌러 평균이 어떻게 변하는지 본다.

17.5. 여기서 배우는 C# 개념

제네릭 <T>: 타입을 변수처럼

List<int>, List<string>, List<Mood> --- 같은 List 클래스이지만 담는 타입이 다르다. List<T> 안에서 T는 “무엇이든 들어올 수 있는 자리”다. 컴파일 시점에 결정되며, List<int>.Add("hi")처럼 잘못된 타입은 컴파일 에러로 즉시 잡힌다. 이를 타입 안전성(type safety)이라 한다. 옛날 ArrayList는 object로 받아 런타임에야 오류가 드러났다. 제네릭은 그 문제를 해결했다.

LINQ: 컬렉션을 “질의”하는 한 줄들

using System.Linq;만 추가하면, 모든 IEnumerable<T>에 마법 같은 메서드가 붙는다.

- .Sum(), .Average(), .Max(), .Min(), .Count()
- .Where(predicate) --- 조건으로 거른다
- .Select(transform) --- 각 원소를 변환
- .OrderBy(key) --- 정렬
- .GroupBy(key) --- 그룹화

이 메서드들은 모두 함수형 스타일이다. for를 직접 쓰는 것보다 “무엇을 원하는가”가 한눈에 보인다.

람다(=>)와 함께 `scores.Where(x => x >= 80)`에서 `x => x >= 80`은 람다 식. “x를 받아 x >= 80을 반환하는 익명 함수”다. 함수를 변수처럼 인자로 넘긴다. 이 자리는 `Func<int, bool>` 타입을 받는다.

`x => x >= 80`이 `Func<int, bool>`이 되는 과정

이 짧은 식 한 줄에 많은 일이 일어나고 있다. 풀어 보자.

1. **Where**는 무엇을 받나? `List<int>`의 **Where**는 다음 시그니처를 가진 메서드다:

```
1 IEnumerable<int> Where(Func<int, bool> predicate);
```

즉 “int를 받아 bool을 반환하는 함수”를 인자로 요구한다.

2. 우리가 줄 수 있는 것 다음 두 가지 모두 `Func<int, bool>`을 만든다:

```
1 // Old way: define a separate method, pass as method group
2 bool IsHigh(int x) { return x >= 80; }
3 scores.Where(IsHigh);
4
5 // New way: write the lambda directly as an argument
6 scores.Where(x => x >= 80);
```

람다 `x => x >= 80`은 “x를 받아 x >= 80을 반환하는 함수”. 컴파일러는 “Where가 `Func<int, bool>`을 원한다”는 것을 알고, 람다의 x를 int로, 반환 타입을 bool로 자동 추론한다.

```
1 x => x + 1           // same as (int x) => x + 1
2 () => 42             // no parameters
3 (a, b) => a + b      // two parameters
4 x => { Debug.Log(x); } // block body (multiple lines)
```

매개변수가 하나일 때만 괄호를 생략할 수 있다.

`using System.Linq;`이 하는 일 --- 확장 메서드

`List<int>` 클래스의 코드를 직접 들여다보면 `Sum`, `Where`, `Max` 같은 메서드는 없다. 그런데 `using System.Linq;`만 써 주면 갑자기 사용 가능해진다. 어떻게?

C#에는 확장 메서드(extension method)라는 기능이 있다. 다른 클래스에 밖에서 메서드를 “덧붙이는” 기법.

```
1 namespace System.Linq
2 {
3     public static class Enumerable
4     {
5         public static int Sum(this IEnumerable<int> source) { ... }
6     }
7 }
```

핵심은 첫 매개변수의 `this` 키워드. 이 표시 덕분에 `scores.Sum()`이라고 쓰면 마치 `List<int>`의 메서드인 것처럼 호출된다.

이는 `using`으로 “보이게”만 한 것이지, `List<int>` 자체는 손대지 않았다. `using System.Linq;`를 빼면 `Sum`, `Where` 등이 보이지 않는다.

실험해 보기

- `scores.OrderByDescending(x => x).Take(3)`으로 상위 3개만 출력해 보자.
- `scores.Select(x => x * 2)`는 무엇을 만들까? 그 결과를 `.ToList()`로 새 List로 변환해 보자.
- `List<int>`을 `List<string>`으로 바꾸고 (`Add("apple")`, `Add("banana")` 등) `.OrderBy(s => s.Length)`로 글자 수 순서로 정렬해 보자. 같은 LINQ 메서드가 어떤 타입에든 통한다는 점이 핵심.

18. Lab 6 --- Coroutine과 람다

3. 람다의 다양한 모양

18.1. 문제

“매 1초마다 무언가를 한다”라는 시간이 흐르는 절차를 코드로 표현하고 싶다. Update에서 타이머를 직접 굴리는 방식은 동작은 하지만 의도가 흐려진다.

18.2. 콘솔이라면 비슷하게

콘솔 C#에서도 `IEnumerable`와 `yield return`을 통해 “값을 하나씩 돌려주는 함수”를 만들 수 있다.

```

1 static IEnumerable<int> CountUp(int start)
2 {
3     int n = start;
4     while (true)
5     {
6         yield return n;    // pause and return n; resume next time
7         n++;
8     }
9 }
10
11 static void Main()
12 {
13     foreach (int v in CountUp(1).Take(5))
14         Console.WriteLine(v);    // 1 2 3 4 5
15 }

```

Unity의 Coroutine은 이 `yield` 메커니즘을 “매 프레임 시간 흐름”에 적용한 것이다.

18.3. Unity 데모

C 키를 누르면 Tick 코루틴을 시작/중지. 1초마다 카운트가 올라가고, 콜백으로 그 값을 받아 출력한다.

```

1 using System;
2 using System.Collections;
3 using UnityEngine;
4
5 public class Lab6_Coroutine : MonoBehaviour
6 {
7     [SerializeField] private float interval = 1f;
8
9     private Coroutine running;

```



```

10
11 void Update()
12 {
13     if (Input.GetKeyDown(KeyCode.C))
14     {
15         if (running == null)
16         {
17             // Lambda: pass a function as an argument
18             running = StartCoroutine(Tick(interval, n => Debug.Log($"Tick #{n}")));
19             Debug.Log("Started.");
20         }
21         else
22         {
23             StopCoroutine(running);
24             running = null;
25             Debug.Log("Stopped.");
26         }
27     }
28 }
29
30 IEnumerator Tick(float wait, Action<int> onTick)
31 {
32     int count = 0;
33     while (true)
34     {
35         yield return new WaitForSeconds(wait);
36         count++;
37         onTick(count); // invoke the passed-in function
38     }
39 }
40 }

```

18.4. Unity 구현 가이드

1. Lab6 GameObject 만들기

Hierarchy → Create Empty → 이름 Lab6.

2. 스크립트 작성 & 부착

Lab6_Coroutine.cs를 작성하고 Lab6에 드래그.

3. Inspector에서 Interval 조정

Inspector에 Interval = 1이 보인다. 0.5로 바꾸면 0.5초마다, 2로 바꾸면 2초마다 카운트 한다. Play 중에도 변경 가능하지만, Play 종료 시점에 원래 값으로 되돌아간다는 점에 주의.

4. 재생

Play → C 키. Console에 Started. 가 출력되고, 1초마다 Tick #1, Tick #2...가 누적된다. 다시 C 키를 누르면 Stopped. 가 찍히고 카운트가 멈춘다.

5. 관찰 포인트

Tick이 도는 동안에도 Unity의 다른 Update는 정상 동작한다. Coroutine은 “양보(yield)”와 “재개”를 반복할 뿐, 멀티스레드가 아니다.

18.5. 여기서 배우는 C# 개념

IEnumerator와 yield return

yield return X를 만나면 메서드의 실행이 중단되고, X가 호출자에게 “돌려진다”. 다음에 호출자가 “계속해 줘”라고 요청하면 중단된 곳부터 다시 실행된다.

Unity의 StartCoroutine은 매 프레임 “계속해 줘”라고 호출하면서, yield return이 돌려준 객체 (예: WaitForSeconds(1f))에 따라 언제 재개할지를 결정한다.

중요: 코루틴은 멀티스레드가 아니다. 단일 스레드에서 “양보(yield)”와 “재개”를 반복할 뿐이다.

람다와 함수 인자

`n => Debug.Log($"Tick #{n}")`는 이름 없는 함수 (람다 식)다.

- `n =>`: “n을 받는다”
- `Debug.Log(...)`: “이걸 하고 void를 돌려준다”

이 람다는 `Action<int>` 타입에 그대로 들어간다. 함수를 값처럼 변수에 담거나, 인자로 넘기는 사고방식이 함수형 프로그래밍의 핵심 한 줄이다.

함수를 인자로 넘기는 “콜백(callback)” 패턴

Tick 메서드의 시그니처를 다시 보자:

```
1 IEnumerator Tick(float wait, Action<int> onTick)
2 {
3     ...
4     onTick(count); // call the function we were given
5 }
```

Tick은 “매 wait초마다 카운트하면서, 카운트 값을 누군가에게 보고한다”. 누구에게 보고하는지는 호출자가 결정한다 --- 인자로 넘긴 `onTick`이 그 “누구”다.

이런 “나중에 호출해 줘”라는 의미의 함수 인자를 콜백(callback)이라 부른다. `Action<T>`, `Func<T, R>` 모두 콜백을 표현하는 타입이다.

콜백을 분리하면 무엇이 좋은가? Tick 안에 `Debug.Log($"Tick {count}")`라고 박혀 있으면, “화면 텍스트 갱신”이나 “사운드 재생”으로 바꾸려면 Tick 자체를 수정해야 한다.

콜백으로 넘기면 Tick은 그대로, “무엇을 할지”만 호출자가 람다로 정한다:

```
1 StartCoroutine(Tick(1f, n => Debug.Log($"Tick {n}"))); // console
2 StartCoroutine(Tick(1f, n => labelText.text = $"#{n}")); // UI
3 StartCoroutine(Tick(1f, n => audio.Play())); // sound
```

Lab 4의 event와 비슷한 “책임 분리” 사상이 콜백에도 흐른다.

yield return이 돌려주는 “객체”의 종류

Unity Coroutine은 yield return이 무엇을 돌려주느냐에 따라 다음에 언제 재개할지를 결정한다. 자주 쓰이는 모양들:

- `yield return null;` --- 다음 프레임에 재개. 가장 자주 씀.
- `yield return new WaitForSeconds(t);` --- t초 후 재개. 게임 시간(Time.timeScale 영향 받음).

- `yield return new WaitForSecondsRealtime(t);` --- 실제 시간 `t`초 후 재개. `timeScale` 영향 안 받음.
- `yield return new WaitUntil(() => 조건);` --- 조건이 `true`가 될 때까지 대기.
- `yield return new WaitForEndOfFrame();` --- 그 프레임의 모든 렌더링이 끝난 후 재개.

`yield return`이 마치 “편지”라면, 우리가 적은 내용에 따라 Unity가 “언제 답장 줄지”를 결정하는 셈이다.

실험해 보기

- `interval`을 `0.5f`로 줄이면 더 빨리 카운트한다. `Time.timeScale = 0.1f`로도 바꿔 보자.
- Tick을 호출할 때 콜백을 두 가지로 분기해 보자: 짝수일 때만 `Debug.LogWarning`, 홀수일 때 `Debug.Log`. 람다 안에 `if`를 넣어도 좋다.
- `yield return new WaitForSeconds(wait)`을 `yield return null`로 바꿔 보자. 매 프레임마다 콜백이 호출되어 “카운트가 미친 듯이 빠르게” 올라간다. “`yield return`이 무엇을 돌려주느냐에 따라 재개 시점이 결정된다”는 원리를 체감.

19. 이번 강의에서 배운 C# 개념 정리

19.1. 한눈에 보기

C# 기능	Lab
<code>enum + switch</code> 식	Lab 1 (큐브 색)
프로퍼티, <code>=></code>	Lab 2 (HP)
<code>interface</code>	Lab 3 (도형)
<code>event / Action<T></code>	Lab 4 (Counter)
제네릭 + LINQ + 람다	Lab 5 (점수)
<code>Coroutine</code> + 람다	Lab 6 (Tick)

19.2. Unity는 우리에게 무엇을 해 주었는가?

이번 강의의 모든 Lab은 콘솔 C#으로도 만들 수 있었다. 하지만 Unity 위에서 동작시키니 다음이 좋아졌다.

- `enum`이 큐브 색으로 바뀌어 보인다.
- `event`가 소리/색/위치로 동시에 반응하는 것을 눈으로 본다.
- `Coroutine`이 시간을 두고 화면에서 펼쳐지는 것을 본다.
- 키보드 입력으로 지금 당장 실험할 수 있다.

C#은 본질이고, Unity는 그 본질을 더 잘 보여 주는 무대다. 이 강의의 핵심 자세다.

20. 단계별 도전 과제

20.1. Phase 1 --- 같은 Lab에 기능 한 줄 더

과제 1.1

Lab 1에 `Mood.Excited`를 추가하고, 키 4를 눌렀을 때 큐브가 무지개색으로 천천히 변하게 하라. 힌트: `Color.HSVToRGB(t, 1, 1), Time.time`.

과제 1.2

Lab 2에 HpBar UI(슬라이더)를 추가하고, HP가 변할 때 슬라이더가 따라 움직이게 하라. Lab 4의 이벤트 패턴을 적용해, `Lab2_Property`에 `event Action<int> OnHpChanged`를 추가하고 HpBar가 그것을 구독하게 한다. Lab 2와 Lab 4를 결합하는 연습이다.

20.2. Phase 2 --- 두 Lab을 합치기

과제 2.1: “작은 인벤토리”

`IItem` 인터페이스(`string Name; int Value;`)와 `List<IItem>` 인벤토리를 만든다. `Sword`, `Potion`, `Key` 등 여러 구현 클래스를 만들고, LINQ로 “가장 비싼 아이템”, “Potion 종류만 필터링” 같은 질의를 해 보자.

Lab 3 + Lab 5의 결합. 아이템 추가 시에도 인벤토리 코드는 한 줄도 바뀌지 않아야 한다.

과제 2.2: “상태 기계”

`enum CharacterState { Idle, Walking, Running, Jumping }`로 캐릭터 상태를 정의하고, 상태가 바뀔 때마다 `event Action<CharacterState> OnStateChanged`를 발생시킨다. 큐브 색이 상태별로 바뀌는 `StateColorView`와, 상태 이름을 화면에 출력하는 `StateLabelView`를 별도로 작성. Lab 1 + Lab 4의 결합.

20.3. Phase 3 --- 자유 실험

과제 3: “내가 가장 헛갈리는 C# 기능 한 가지를 Lab으로”

이 강의의 6개 Lab은 시작점일 뿐이다. 자신이 잘 이해되지 않는 C# 기능 하나를 골라 스스로 Lab 하나를 설계하고 만들어 보자.

추천 주제:

- `struct` vs `class`: 값 복사 vs 참조 복사 (큐브 두 개로 시연)
- `out/ref` 파라미터
- nullable 타입 `int?`와 ??
- `abstract class` vs `interface`의 차이
- `static` 메서드/필드
- 예외 처리 `try/catch/finally`

문제 정의 → 콘솔 코드 → Unity 데모 → 실험 과제, 이 흐름을 본인이 작성해 본다. “누군가에게 설명하는 것”만큼 자신이 잘 이해했는지 확인하는 빠른 길은 없다.

21. 이번 강의에서 배운 핵심 개념

- 이 강의의 주인공은 C#이다. Unity는 **Console.WriteLine**보다 훨씬 친절한 무대다.
- **enum + switch** 식은 “상태”를 안전하고 명확하게 표현하게 해 준다.
- 프로퍼티는 “외부 접근의 범위”를 한 줄로 조절하는 강력한 도구다.
- 인터페이스는 “공통 능력”만 약속하여, 새로운 종류가 추가되어도 호출 코드를 바꾸지 않게 해 준다.
- **event/Action<T>**는 발신자와 수신자를 이름으로 묶지 않고 연결한다.
- 제네릭과 LINQ는 컬렉션을 타입 안전하게 + 함수형 스타일로 다루게 해 준다.
- **Coroutine**과 람다는 “시간 흐름”과 “함수를 값으로” 다루는 두 가지 강력한 사고방식이다.

다음 시간엔?

이번 강의의 6개 Lab을 응용해, “C# 기능 두세 가지를 한꺼번에 쓰는 작은 프로젝트”로 발전시킨다. 도전 과제 Phase 2가 그 시작점이다. 자신이 만든 Lab을 정리해 두면, 학기말 발표에서 좋은 자료가 된다.